



Recent advances in convolutional neural networks



Jiuxiang Gu^{a,*}, Zhenhua Wang^{b,1}, Jason Kuen^b, Lianyang Ma^b, Amir Shahroudy^b,
Bing Shuai^b, Ting Liu^b, Xingxing Wang^b, Gang Wang^b, Jianfei Cai^c, Tsuhan Chen^c

^a ROSE Lab, Interdisciplinary Graduate School, Nanyang Technological University, Singapore

^b School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore

^c School of Computer Science and Engineering, Nanyang Technological University, Singapore

ARTICLE INFO

Article history:

Received 12 August 2016

Revised 26 September 2017

Accepted 10 October 2017

Available online 11 October 2017

Keywords:

Convolutional neural network

Deep learning

ABSTRACT

In the last few years, deep learning has led to very good performance on a variety of problems, such as visual recognition, speech recognition and natural language processing. Among different types of deep neural networks, convolutional neural networks have been most extensively studied. Leveraging on the rapid growth in the amount of the annotated data and the great improvements in the strengths of graphics processor units, the research on convolutional neural networks has been emerged swiftly and achieved state-of-the-art results on various tasks. In this paper, we provide a broad survey of the recent advances in convolutional neural networks. We detailize the improvements of CNN on different aspects, including layer design, activation function, loss function, regularization, optimization and fast computation. Besides, we also introduce various applications of convolutional neural networks in computer vision, speech and natural language processing.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

Convolutional Neural Network (CNN) is a well-known deep learning architecture inspired by the natural visual perception mechanism of the living creatures. In 1959, Hubel & Wiesel [1] found that cells in animal visual cortex are responsible for detecting light in receptive fields. Inspired by this discovery, Kuniyiko Fukushima proposed the neocognitron in 1980 [2], which could be regarded as the predecessor of CNN. In 1990, LeCun et al. [3] published the seminal paper establishing the modern framework of CNN, and later improved it in [4]. They developed a multi-layer artificial neural network called LeNet-5 which could classify handwritten digits. Like other neural networks, LeNet-5 has multiple layers and can be trained with the backpropagation algorithm [5]. It can obtain effective representations of the original image, which makes it possible to recognize visual patterns directly from raw pixels with little-to-none preprocessing. A parallel study of Zhang et al. [6] used a shift-invariant artificial neural network (SIANN) to recognize characters from an image. However,

due to the lack of large training data and computing power at that time, their networks can not perform well on more complex problems, e.g., large-scale image and video classification.

Since 2006, many methods have been developed to overcome the difficulties encountered in training deep CNNs [7–10]. Most notably, Krizhevsky et al. proposed a classic CNN architecture and showed significant improvements upon previous methods on the image classification task. The overall architecture of their method, i.e., AlexNet [8], is similar to LeNet-5 but with a deeper structure. With the success of AlexNet, many works have been proposed to improve its performance. Among them, four representative works are ZFNet [11], VGGNet [9], GoogleNet [10] and ResNet [12]. From the evolution of the architectures, a typical trend is that the networks are getting deeper, e.g., ResNet, which won the champion of ILSVRC 2015, is about 20 times deeper than AlexNet and 8 times deeper than VGGNet. By increasing depth, the network can better approximate the target function with increased nonlinearity and get better feature representations. However, it also increases the complexity of the network, which makes the network be more difficult to optimize and easier to get overfitting. Along this way, various methods have been proposed to deal with these problems in various aspects. In this paper, we try to give a comprehensive review of recent advances and give some thorough discussions.

In the following sections, we identify broad categories of works related to CNN. Fig. 1 shows the hierarchically-structured taxonomy of this paper. We first give an overview of the basic compo-

* Corresponding author.

E-mail addresses: jgu004@ntu.edu.sg (J. Gu), jasonkuen@ntu.edu.sg (J. Kuen), lyma@ntu.edu.sg (L. Ma), amir3@ntu.edu.sg (A. Shahroudy), bshuai001@ntu.edu.sg (B. Shuai), LIUT0016@e.ntu.edu.sg (T. Liu), wangxx@ntu.edu.sg (X. Wang), WangGang@ntu.edu.sg (G. Wang), asjfc@ntu.edu.sg (J. Cai), tsuhan@ntu.edu.sg (T. Chen).

¹ equal contribution

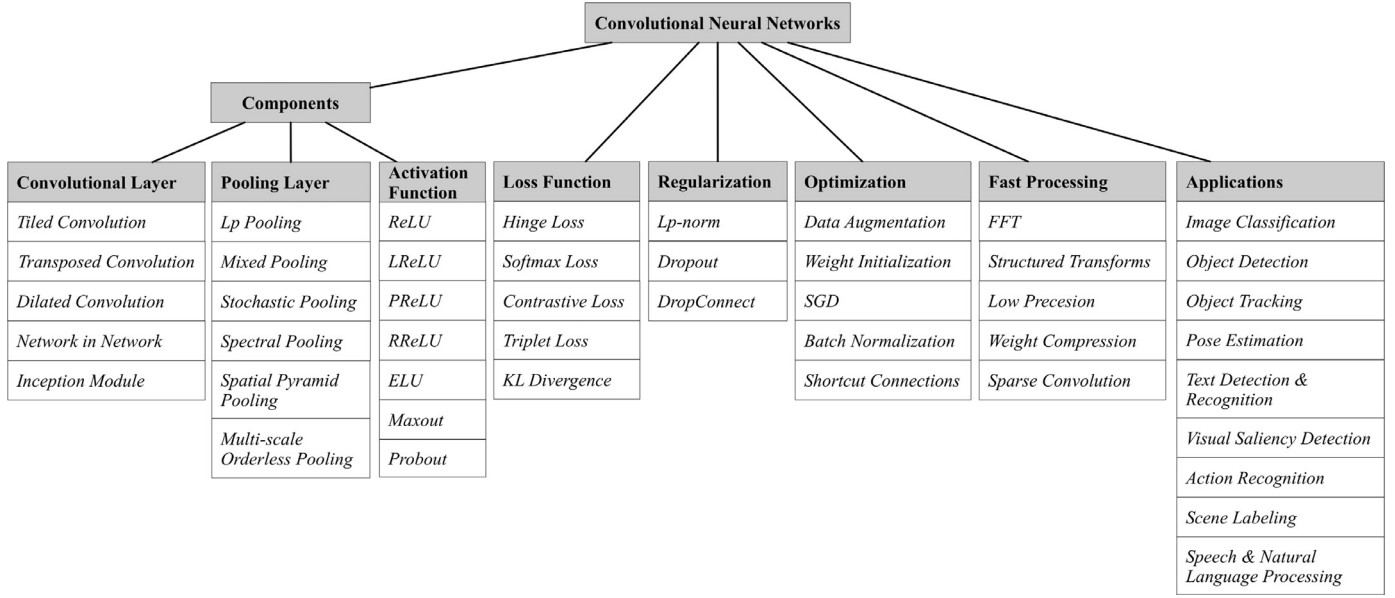


Fig. 1. Hierarchically-structured taxonomy of this survey.

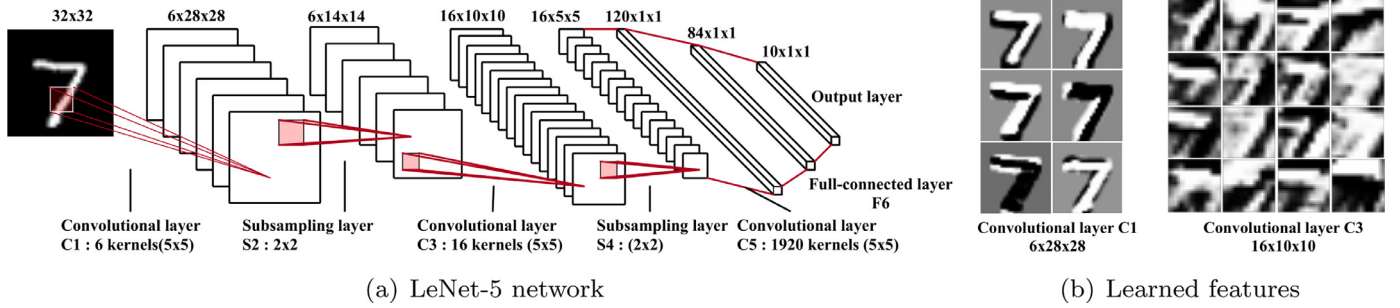


Fig. 2. (a) The architecture of the LeNet-5 network, which works well on digit classification task. (b) Visualization of features in the LeNet-5 network. Each layer's feature maps are displayed in a different block.

nents of CNN in Section 2. Then, we introduce some recent improvements on different aspects of CNN including convolutional layer, pooling layer, activation function, loss function, regularization and optimization in Section 3 and introduce the fast computing techniques in Section 4. Next, we discuss some typical applications of CNN including image classification, object detection, object tracking, pose estimation, text detection and recognition, visual saliency detection, action recognition, scene labeling, speech and natural language processing in Section 5. Finally, we conclude this paper in Section 6.

2. Basic CNN components

There are numerous variants of CNN architectures in the literature. However, their basic components are very similar. Taking the famous LeNet-5 as an example, it consists of three types of layers, namely convolutional, pooling, and fully-connected layers. The convolutional layer aims to learn feature representations of the inputs. As shown in Fig. 2(a), convolution layer is composed of several convolution kernels which are used to compute different feature maps. Specifically, each neuron of a feature map is connected to a region of neighbouring neurons in the previous layer. Such a neighbourhood is referred to as the neuron's receptive field in the previous layer. The new feature map can be obtained by first convolving the input with a learned kernel and then applying an element-wise nonlinear activation function on the convolved results. Note that, to generate each feature map, the kernel is shared by all spatial

locations of the input. The complete feature maps are obtained by using several different kernels. Mathematically, the feature value at location (i, j) in the k th feature map of l th layer, $z_{i,j,k}^l$, is calculated by:

$$z_{i,j,k}^l = \mathbf{w}_k^l \mathbf{x}_{i,j}^l + b_k^l \quad (1)$$

where $\mathbf{w}_k^l$ and b_k^l are the weight vector and bias term of the k th filter of the l th layer respectively, and $\mathbf{x}_{i,j}^l$ is the input patch centered at location (i, j) of the l th layer. Note that the kernel $\mathbf{w}_k^l$ that generates the feature map $\mathbf{z}_{i,j,k}^l$ is shared. Such a weight sharing mechanism has several advantages such as it can reduce the model complexity and make the network easier to train. The activation function introduces nonlinearities to CNN, which are desirable for multi-layer networks to detect nonlinear features. Let $a(\cdot)$ denote the nonlinear activation function. The activation value $a_{i,j,k}^l$ of convolutional feature $z_{i,j,k}^l$ can be computed as:

$$a_{i,j,k}^l = a(z_{i,j,k}^l) \quad (2)$$

Typical activation functions are sigmoid, tanh [13] and ReLU [14]. The pooling layer aims to achieve shift-invariance by reducing the resolution of the feature maps. It is usually placed between two convolutional layers. Each feature map of a pooling layer is connected to its corresponding feature map of the preceding convolutional layer. Denoting the pooling function as $\text{pool}(\cdot)$,

for each feature map $\mathbf{a}_{i,j,k}^l$ we have:

$$y_{i,j,k}^l = \text{pool}(\mathbf{a}_{m,n,k}^l), \forall (m, n) \in \mathcal{R}_{ij} \quad (3)$$

where $\mathcal{R}_{ij}$ is a local neighbourhood around location (i, j) . The typical pooling operations are average pooling [15] and max pooling [16]. Fig. 2(b) shows the feature maps of digit 7 learned by the first two convolutional layers. The kernels in the 1st convolutional layer are designed to detect low-level features such as edges and curves, while the kernels in higher layers are learned to encode more abstract features. By stacking several convolutional and pooling layers, we could gradually extract higher-level feature representations.

After several convolutional and pooling layers, there may be one or more fully-connected layers which aim to perform high-level reasoning [9,11,17]. They take all neurons in the previous layer and connect them to every single neuron of current layer to generate global semantic information. Note that fully-connected layer not always necessary as it can be replaced by a 1×1 convolution layer [18].

The last layer of CNNs is an output layer. For classification tasks, the softmax operator is commonly used [8]. Another commonly used method is SVM, which can be combined with CNN features to solve different classification tasks [19,20]. Let θ denote all the parameters of a CNN (e.g., the weight vectors and bias terms). The optimum parameters for a specific task can be obtained by minimizing an appropriate loss function defined on that task. Suppose we have N desired input-output relations $\{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}; n \in [1, \dots, N]\}$, where $\mathbf{x}^{(n)}$ is the n -th input data, $\mathbf{y}^{(n)}$ is its corresponding target label and $\mathbf{o}^{(n)}$ is the output of CNN. The loss of CNN can be calculated as follows:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \ell(\theta; \mathbf{y}^{(n)}, \mathbf{o}^{(n)}) \quad (4)$$

Training CNN is a problem of global optimization. By minimizing the loss function, we can find the best fitting set of parameters. Stochastic gradient descent is a common solution for optimizing CNN network [21,22].

3. Improvements on CNNs

There have been various improvements on CNNs since the success of AlexNet in 2012. In this section, we describe the major improvements on CNNs from six aspects: convolutional layer, pooling layer, activation function, loss function, regularization, and optimization.

3.1. Convolutional layer

Convolution filter in basic CNNs is a generalized linear model (GLM) for the underlying local image patch. It works well for abstraction when instances of latent concepts are linearly separable. Here we introduce some works which aim to enhance its representation ability.

3.1.1. Tiled convolution

Weight sharing mechanism in CNNs can drastically decrease the number of parameters. However, it may also restrict the models from learning other kinds of invariance. Tiled CNN [23] is a variation of CNN that tiles and multiples feature maps to learn rotational and scale invariant features. Separate kernels are learned within the same layer, and the complex invariances can be learned implicitly by square-root pooling over neighbouring units. As illustrated in Fig. 3(b), the convolution operations are applied every k unit, where k is the tile size to control the distance over which weights are shared. When the tile size k is 1, the units within each

map will have the same weights, and tiled CNN becomes identical to the traditional CNN. In [23], their experiments on the NORB and CIFAR-10 datasets show that $k=2$ achieves the best results. Wang et al. [24] find that Tiled CNN performs better than traditional CNN [25] on small time series datasets.

3.1.2. Transposed convolution

Transposed convolution can be seen as the backward pass of a corresponding traditional convolution. It is also known as deconvolution [11,26–28] and fractionally strided convolution [29]. To stay consistent with most literature [11,30], we use the term “deconvolution”. Contrary to the traditional convolution that connects multiple input activations to a single activation, deconvolution associates a single activation with multiple output activations. Fig. 3(d) shows a deconvolution operation of 3×3 kernel over a 4×4 input using unit stride and zero padding. The stride of deconvolution gives the dilation factor for the input feature map. Specifically, the deconvolution will first upsample the input by a factor of the stride value with padding, then perform convolution operation on the upsampled input. Recently, deconvolution has been widely used for visualization [11], recognition [31–33], localization [34], semantic segmentation [30], visual question answering [35], and super-resolution [36].

3.1.3. Dilated convolution

Dilated CNN [37] is a recent development of CNN that introduces one more hyper-parameter to the convolutional layer. By inserting zeros between filter elements, Dilated CNN can increase the network's receptive field size and let the network cover more relevant information. This is very important for tasks which need a large receptive field when making the prediction. Formally, a 1-D dilated convolution with dilation l that convolves signal F with kernel k of size r is defined as $(F *_l k)_t = \sum_{\tau} k_{\tau} F_{t-l\tau}$, where $*_l$ denotes l -dilated convolution. This formula can be straightforwardly extended to 2-D dilated convolution. Fig. 3(c) shows an example of three dilated convolution layers where the dilation factor l grows up exponentially at each layer. The middle feature map F_2 is produced from the bottom feature map F_1 by applying a 1-dilated convolution, where each element in F_2 has a receptive field of 3×3 . F_3 is produced from F_2 by applying a 2-dilated convolution, where each element in F_3 has a receptive field of $(2^3 - 1) \times (2^3 - 1)$. The top feature map F_4 is produced from F_3 by applying a 4-dilated convolution, where each element in F_4 has a receptive field of $(2^4 - 1) \times (2^4 - 1)$. As can be seen, the size of receptive field of each element in F_{i+1} is $(2^{i+2} - 1) \times (2^{i+2} - 1)$. Dilated CNNs have achieved impressive performance in tasks such as scene segmentation [37], machine translation [38], speech synthesis [39], and speech recognition [40].

3.1.4. Network in network

Network In Network (NIN) is a general network structure proposed by Lin et al. [18]. It replaces the linear filter of the convolutional layer by a micro network, e.g., multilayer perceptron convolution (mlpconv) layer in the paper, which makes it capable of approximating more abstract representations of the latent concepts. The overall structure of NIN is the stacking of such micro networks. Fig. 4 shows the difference between the linear convolutional layer and the mlpconv layer. Formally, the feature map of convolution layer (with nonlinear activation function, e.g., ReLU [14]) is computed as:

$$a_{i,j,k} = \max(\mathbf{w}_k^T \mathbf{x}_{i,j} + b_k, 0) \quad (5)$$

where $a_{i,j,k}$ is the activation value of k th feature map at location (i, j) , $\mathbf{x}_{i,j}$ is the input patch centered at location (i, j) , $\mathbf{w}_k$ and b_k are weight vector and bias term of the k th filter. As a comparison, the

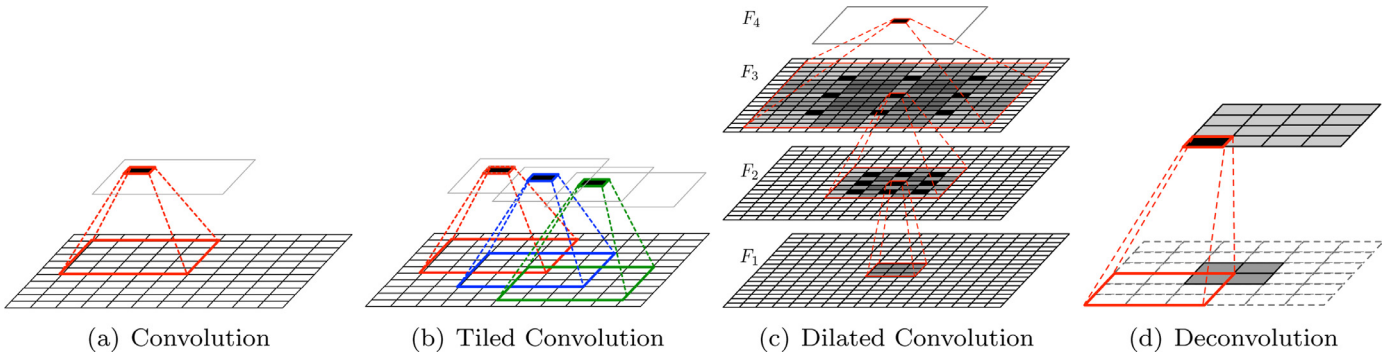


Fig. 3. Illustration of (a) Convolution, (b) Tiled Convolution, (c) Dilated Convolution, and (d) Deconvolution.

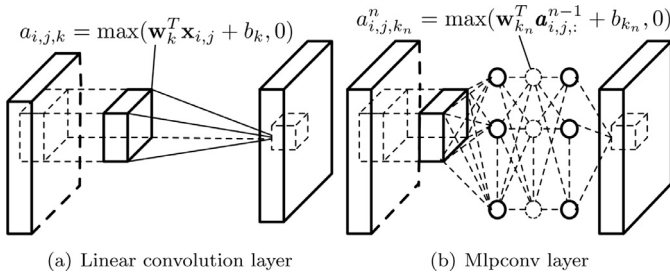


Fig. 4. The comparison of linear convolution layer and mlpconv layer.

computation performed by mlpconv layer is formulated as:

$$a_{i,j,k_n}^n = \max(\mathbf{w}_{k_n}^T \mathbf{a}_{i,j,:}^{n-1} + b_{k_n}, 0) \quad (6)$$

where $n \in [1, N]$, N is the number of layers in the mlpconv layer, $\mathbf{a}_{i,j,:}^0$ is equal to $\mathbf{x}_{i,j}$. In mlpconv layer, 1×1 convolutions are placed after the traditional convolutional layer. The 1×1 convolution is equivalent to the cross-channel parametric pooling operation which is succeeded by ReLU [14]. Therefore, the mlpconv layer can also be regarded as the cascaded cross-channel parametric pooling on the normal convolutional layer. In the end, they also apply a global average pooling which spatially averages the feature maps of the final layer, and directly feed the output vector into softmax layer. Compared with the fully-connected layer, global average pooling has fewer parameters and thus reduces the overfitting risk and computational load.

3.1.5. Inception module

Inception module is introduced by Szegedy et al. [10] which can be seen as a logical culmination of NIN. They use variable filter sizes to capture different visual patterns of different sizes, and approximate the optimal sparse structure by the inception module. Specifically, inception module consists of one pooling operation and three types of convolution operations (see Fig. 5(b)), and 1×1 convolutions are placed before 3×3 and 5×5 convolutions as dimension reduction modules, which allow for increasing the depth and width of CNN without increasing the computational complexity. With the help of inception module, the network parameters can be dramatically reduced to 5 millions which are much less than those of AlexNet (60 millions) and ZFNet (75 millions).

In their later paper [41], to find high-performance networks with a relatively modest computation cost, they suggest the representation size should gently decrease from inputs to outputs as well as spatial aggregation can be done over lower dimensional embeddings without much loss in representational power. The optimal performance of the network can be reached by balancing the number of filters per layer and the depth of the network. Inspired by the ResNet [12], their latest Inception-V4 [42] combines

the inception architecture with shortcut connections (see Fig. 5(d)). They find that shortcut connections can significantly accelerate the training of inception networks. Their Inception-v4 model architecture (with 75 trainable layers) that ensembles three residual and one Inception-v4 can achieve 3.08% top-5 error rate on the validation dataset of ILSVRC 2012.

3.2. Pooling layer

Pooling is an important concept of CNN. It lowers the computational burden by reducing the number of connections between convolutional layers. In this section, we introduce some recent pooling methods used in CNNs.

3.2.1. L_p Pooling

L_p pooling is a biologically inspired pooling process modelled on complex cells [43]. It has been theoretically analyzed in [44], which suggest that L_p pooling provides better generalization than max pooling. L_p pooling can be represented as:

$$y_{i,j,k} = \left[\sum_{(m,n) \in \mathcal{R}_{ij}} (a_{m,n,k})^p \right]^{1/p} \quad (7)$$

where $y_{i,j,k}$ is the output of the pooling operator at location (i, j) in k th feature map, and $a_{m,n,k}$ is the feature value at location (m, n) within the pooling region $\mathcal{R}_{ij}$ in k th feature map. Specially, when $p = 1$, L_p corresponds to average pooling, and when $p = \infty$, L_p reduces to max pooling.

3.2.2. Mixed pooling

Inspired by random Dropout [17] and DropConnect [45], Yu et al. [46] propose a mixed pooling method which is the combination of max pooling and average pooling. The function of mixed pooling can be formulated as follows:

$$y_{i,j,k} = \lambda \max_{(m,n) \in \mathcal{R}_{ij}} a_{m,n,k} + (1 - \lambda) \frac{1}{|\mathcal{R}_{ij}|} \sum_{(m,n) \in \mathcal{R}_{ij}} a_{m,n,k} \quad (8)$$

where λ is a random value being either 0 or 1 which indicates the choice of either using average pooling or max pooling. During forward propagation process, λ is recorded and will be used for the backpropagation operation. Experiments in [46] show that mixed pooling can better address the overfitting problems and it performs better than max pooling and average pooling.

3.2.3. Stochastic pooling

Stochastic pooling [47] is a dropout-inspired pooling method. Instead of picking the maximum value within each pooling region as max pooling does, stochastic pooling randomly picks the activations according to a multinomial distribution, which ensures

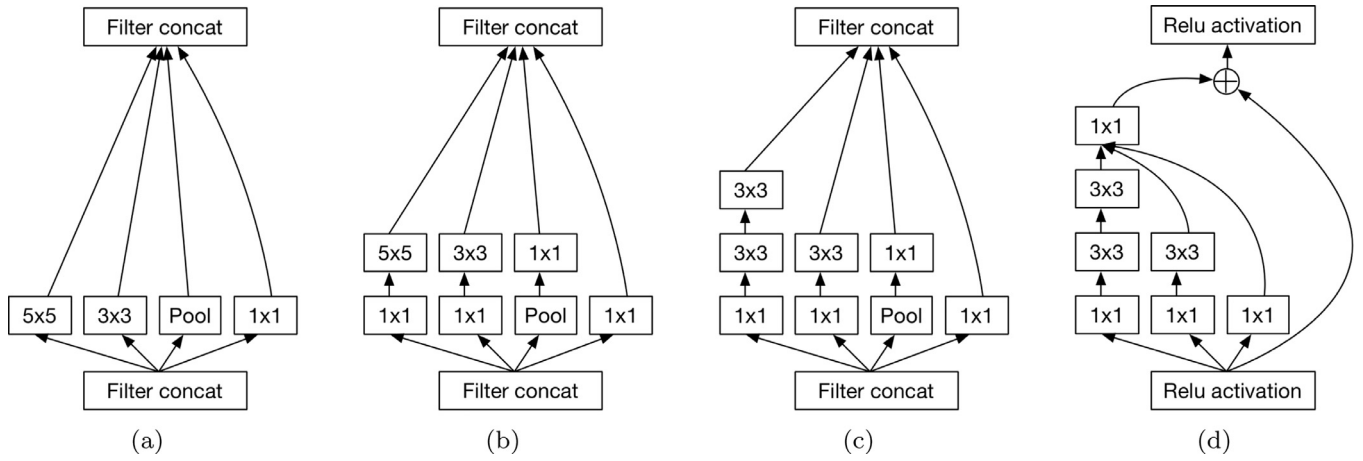


Fig. 5. (a) Inception module, naive version. (b) The inception module used in [10]. (c) The improved inception module used in [41] where each 5×5 convolution is replaced by two 3×3 convolutions. (d) The Inception-ResNet-A module used in [42].

that the non-maximal activations of feature maps are also possible to be utilized. Specifically, stochastic pooling first computes the probabilities p for each region R_j by normalizing the activations within the region, i.e., $p_i = a_i / \sum_{k \in R_j} (a_k)$. After obtaining the distribution $P(p_1, \dots, p_{|R_j|})$, we can sample from the multinomial distribution based on p to pick a location l within the region, and then set the pooled activation as $y_j = a_l$, where $l \sim P(p_1, \dots, p_{|R_j|})$. Compared with max pooling, stochastic pooling can avoid overfitting due to the stochastic component.

3.2.4. Spectral pooling

Spectral pooling [48] performs dimensionality reduction by cropping the representation of input in frequency domain. Given an input feature map $\mathbf{x} \in \mathbb{R}^{m \times m}$, suppose the dimension of desired output feature map is $h \times w$, spectral pooling first computes the discrete Fourier transform (DFT) of the input feature map, then crops the frequency representation by maintaining only the central $h \times w$ submatrix of the frequencies, and finally uses inverse DFT to map the approximation back into spatial domain. Compared with max pooling, the linear low-pass filtering operation of spectral pooling can preserve more information for the same output dimensionality. Meanwhile, it also does not suffer from the sharp reduction in output map dimensionality exhibited by other pooling methods. What is more, the process of spectral pooling is achieved by matrix truncation, which makes it capable of being implemented with little computational cost in CNNs (e.g., [49]) that employ FFT for convolution kernels.

3.2.5. Spatial pyramid pooling

Spatial pyramid pooling (SPP) is introduced by He et al. [50]. The key advantage of SPP is that it can generate a fixed-length representation regardless of the input sizes. SPP pools input feature map in local spatial bins with sizes proportional to the image size, resulting in a fixed number of bins. This is different from the sliding window pooling in the previous deep networks, where the number of sliding windows depends on the input size. By replacing the last pooling layer with SPP, they propose a new SPP-net which is able to deal with images with different sizes.

3.2.6. Multi-scale orderless pooling

Inspired by [51], Gong et al. [52] use multi-scale orderless pooling (MOP) to improve the invariance of CNNs without degrading their discriminative power. They extract deep activation features for both the whole image and local patches of several scales. The activations of the whole image are the same as those of previous

CNNs, which aim to capture the global spatial layout information. The activations of local patches are aggregated by VLAD encoding [53], which aim to capture more local, fine-grained details of the image as well as enhancing invariance. The new image representation is obtained by concatenating the global activations and the VLAD features of the local patch activations.

3.3. Activation function

A proper activation function significantly improves the performance of a CNN for a certain task. In this section, we introduce the recently used activation functions in CNNs.

3.3.1. ReLU

Rectified linear unit (ReLU) [14] is one of the most notable non-saturated activation functions. The ReLU activation function is defined as:

$$a_{i,j,k} = \max(z_{i,j,k}, 0) \quad (9)$$

where $z_{i,j,k}$ is the input of the activation function at location (i, j) on the k th channel. ReLU is a piecewise linear function which prunes the negative part to zero and retains the positive part (see Fig. 6(a)). The simple $\max(\cdot)$ operation of ReLU allows it to compute much faster than sigmoid or tanh activation functions, and it also induces the sparsity in the hidden units and allows the network to easily obtain sparse representations. It has been shown that deep networks can be trained efficiently using ReLU even without pre-training [8]. Even though the discontinuity of ReLU at 0 may hurt the performance of backpropagation, many works have shown that ReLU works better than sigmoid and tanh activation functions empirically [54,55].

3.3.2. Leaky ReLU

A potential disadvantage of ReLU unit is that it has zero gradient whenever the unit is not active. This may cause units that do not active initially never active as the gradient-based optimization will not adjust their weights. Also, it may slow down the training process due to the constant zero gradients. To alleviate this problem, Mass et al. introduce Leaky ReLU (LReLU) [54] which is defined as:

$$a_{i,j,k} = \max(z_{i,j,k}, 0) + \lambda \min(z_{i,j,k}, 0) \quad (10)$$

where λ is a predefined parameter in range $(0, 1)$. Compared with ReLU, Leaky ReLU compresses the negative part rather than mapping it to constant zero, which makes it allow for a small, non-zero gradient when the unit is not active.

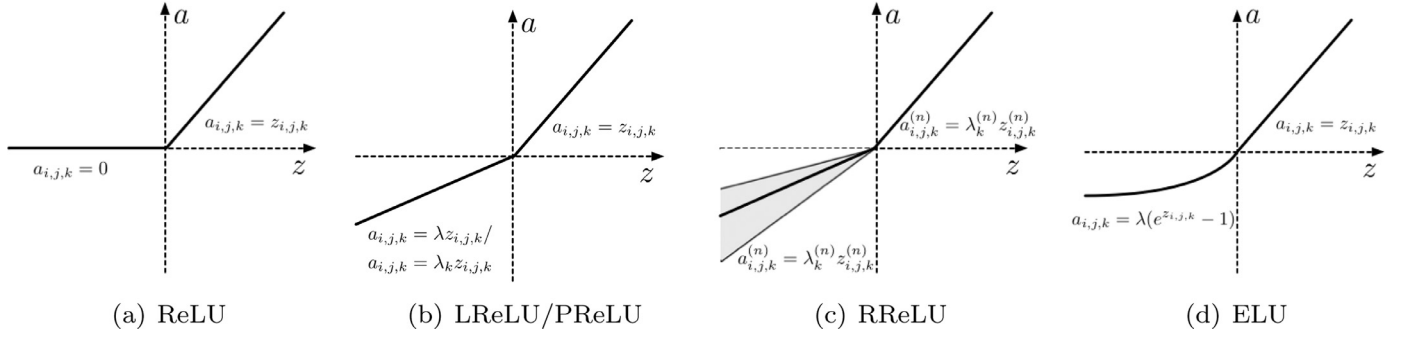


Fig. 6. The comparison among ReLU, LReLU, PReLU, RReLU and ELU. For Leaky ReLU, λ is empirically predefined. For PReLU, λ_k is learned from training data. For RReLU, $\lambda_k^{(n)}$ is a random variable which is sampled from a given uniform distribution in training and keeps fixed in testing. For ELU, λ is empirically predefined.

3.3.3. Parametric ReLU

Rather than using a predefined parameter in Leaky ReLU, e.g., λ in Eq. (10), He et al. [56] propose Parametric Rectified Linear Unit (PReLU) which adaptively learns the parameters of the rectifiers in order to improve accuracy. Mathematically, PReLU function is defined as:

$$a_{i,j,k} = \max(z_{i,j,k}, 0) + \lambda_k \min(z_{i,j,k}, 0) \quad (11)$$

where λ_k is the learned parameter for the k th channel. As PReLU only introduces a very small number of extra parameters, e.g., the number of extra parameters is the same as the number of channels of the whole network, there is no extra risk of overfitting and the extra computational cost is negligible. It also can be simultaneously trained with other parameters by backpropagation.

3.3.4. Randomized ReLU

Another variant of Leaky ReLU is Randomized Leaky Rectified Linear Unit (RReLU) [57]. In RReLU, the parameters of negative parts are randomly sampled from a uniform distribution in training, and then fixed in testing (see Fig. 6(c)). Formally, RReLU function is defined as:

$$a_{i,j,k}^{(n)} = \max(z_{i,j,k}^{(n)}, 0) + \lambda_k^{(n)} \min(z_{i,j,k}^{(n)}, 0) \quad (12)$$

where $z_{i,j,k}^{(n)}$ denotes the input of activation function at location (i, j) on the k th channel of n th example, $\lambda_k^{(n)}$ denotes its corresponding sampled parameter, and $a_{i,j,k}^{(n)}$ denotes its corresponding output. It could reduce overfitting due to its randomized nature. Xu et al. [57] also evaluate ReLU, LReLU, PReLU and RReLU on standard image classification task, and concludes that incorporating a non-zero slop for negative part in rectified activation units could consistently improve the performance.

3.3.5. ELU

Clevert et al. [58] introduce Exponential Linear Unit (ELU) which enables faster learning of deep neural networks and leads to higher classification accuracies. Like ReLU, LReLU, PReLU and RReLU, ELU avoids the vanishing gradient problem by setting the positive part to identity. In contrast to ReLU, ELU has a negative part which is beneficial for fast learning.

Compared with LReLU, PReLU, and RReLU which also have unsaturated negative parts, ELU employs a saturation function as negative part. As the saturation function will decrease the variation of the units if deactivated, it makes ELU more robust to noise. The function of ELU is defined as:

$$a_{i,j,k} = \max(z_{i,j,k}, 0) + \min(\lambda(e^{z_{i,j,k}} - 1), 0) \quad (13)$$

where λ is a predefined parameter for controlling the value to which an ELU saturate for negative inputs.

3.3.6. Maxout

Maxout [59] is an alternative nonlinear function that takes the maximum response across multiple channels at each spatial position. As stated in [59], the maxout function is defined as: $a_{i,j,k} = \max_{k \in [1, K]} z_{i,j,k}$, where $z_{i,j,k}$ is the k th channel of the feature map. It is worth noting that maxout enjoys all the benefits of ReLU since ReLU is actually a special case of maxout, e.g., $\max(\mathbf{w}_1^T \mathbf{x} + b_1, \mathbf{w}_2^T \mathbf{x} + b_2)$ where $\mathbf{w}_1$ is a zero vector and b_1 is zero. Besides, maxout is particularly well suited for training with Dropout.

3.3.7. Probout

Springenberg et al. [60] propose a probabilistic variant of maxout called probout. They replace the maximum operation in maxout with a probabilistic sampling procedure. Specifically, they first define a probability for each of the k linear units as: $p_i = e^{\lambda z_i} / \sum_{j=1}^k e^{\lambda z_j}$, where λ is a hyperparameter for controlling the variance of the distribution. Then, they pick one of the k units according to a multinomial distribution $\{p_1, \dots, p_k\}$ and set the activation value to be the value of the picked unit. In order to incorporate with dropout, they actually re-define the probabilities as:

$$\hat{p}_0 = 0.5, \quad \hat{p}_i = e^{\lambda z_i} / \left(2 \cdot \sum_{j=1}^k e^{\lambda z_j} \right) \quad (14)$$

The activation function is then sampled as:

$$a_i = \begin{cases} 0 & \text{if } i = 0 \\ z_i & \text{else} \end{cases} \quad (15)$$

where $i \sim \text{multinomial}\{\hat{p}_0, \dots, \hat{p}_k\}$. Probout can achieve the balance between preserving the desirable properties of maxout units and improving their invariance properties. However, in testing process, probout is computationally expensive than maxout due to the additional probability calculations.

3.4. Loss function

It is important to choose an appropriate loss function for a specific task. We introduce four representative ones in this subsection: Hinge loss, Softmax loss, Contrastive loss, Triplet loss.

3.4.1. Hinge loss

Hinge loss is usually used to train large margin classifiers such as Support Vector Machine (SVM). The hinge loss function of a multi-class SVM is defined in Eq. (16), where $\mathbf{w}$ is the weight vector of classifier and $\mathbf{y}^{(i)} \in [1, \dots, K]$ indicates its correct class label among the K classes.

$$\mathcal{L}_{\text{hinge}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K [\max(0, 1 - \delta(\mathbf{y}^{(i)}, j) \mathbf{w}^T \mathbf{x}_i)]^p \quad (16)$$

where $\delta(\mathbf{y}^{(i)}, j) = 1$ if $\mathbf{y}^{(i)} = j$, otherwise $\delta(\mathbf{y}^{(i)}, j) = -1$. Note that if $p = 1$, Eq. (16) is Hinge-Loss (L_1 -Loss), while if $p = 2$, it is the Squared Hinge-Loss (L_2 -Loss) [61]. The L_2 -Loss is differentiable and imposes a larger loss for point which violates the margin comparing with L_1 -Loss. Tang [19] investigates and compares the performance of softmax with L_2 -SVMs in deep networks. The results on MNIST [62] demonstrate the superiority of L_2 -SVM over softmax.

3.4.2. Softmax loss

Softmax loss is a commonly used loss function which is essentially a combination of multinomial logistic loss and softmax. Given a training set $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}); i \in 1, \dots, N, \mathbf{y}^{(i)} \in 1, \dots, K\}$, where $\mathbf{x}^{(i)}$ is the i th input image patch, and $\mathbf{y}^{(i)}$ is its target class label among the K classes. The prediction of j th class for i th input is transformed with the softmax function: $p_j^{(i)} = e^{z_j^{(i)}} / \sum_{l=1}^K e^{z_l^{(i)}}$, where $z_j^{(i)}$ is usually the activations of a densely connected layer, so $z_j^{(i)}$ can be written as $z_j^{(i)} = \mathbf{w}_j^T \mathbf{a}^{(i)} + \mathbf{b}_j$. Softmax turns the predictions into non-negative values and normalizes them to get a probability distribution over classes. Such probabilistic predictions are used to compute the multinomial logistic loss, i.e., the softmax loss, as follows:

$$\mathcal{L}_{\text{softmax}} = -\frac{1}{N} \left[\sum_{i=1}^N \sum_{j=1}^K 1\{\mathbf{y}^{(i)} = j\} \log p_j^{(i)} \right] \quad (17)$$

Recently, Liu et al. [63] propose the Large-Margin Softmax (L-Softmax) loss, which introduces an angular margin to the angle θ_j between input feature vector $\mathbf{a}^{(i)}$ and the j th column $\mathbf{w}_j$ of weight matrix. The prediction $p_j^{(i)}$ for L-Softmax loss is defined as:

$$p_j^{(i)} = \frac{e^{\|\mathbf{w}_j\| \|\mathbf{a}^{(i)}\| \psi(\theta_j)}}{e^{\|\mathbf{w}_j\| \|\mathbf{a}^{(i)}\| \psi(\theta_j)} + \sum_{l \neq j} e^{\|\mathbf{w}_l\| \|\mathbf{a}^{(i)}\| \cos(\theta_l)}} \quad (18)$$

$$\psi(\theta_j) = (-1)^k \cos(m\theta_j) - 2k, \theta_j \in [k\pi/m, (k+1)\pi/m] \quad (19)$$

where $k \in [0, m-1]$ is an integer, m controls the margin among classes. When $m = 1$, the L-Softmax loss reduces to the original softmax loss. By adjusting the margin m between classes, a relatively difficult learning objective will be defined, which can effectively avoid overfitting. They verify the effective of L-Softmax on MNIST, CIFAR-10, and CIFAR-100, and find that the L-Softmax loss performs better than the original softmax.

3.4.3. Contrastive loss

Contrastive loss is commonly used to train Siamese network [64–67] which is a weakly-supervised scheme for learning a similarity measure from pairs of data instances labelled as matching or non-matching. Given the i th pair of data $(\mathbf{x}_\alpha^{(i)}, \mathbf{x}_\beta^{(i)})$, let $(\mathbf{z}_\alpha^{(i,l)}, \mathbf{z}_\beta^{(i,l)})$ denotes its corresponding output pair of the l th ($l \in [1, \dots, L]$) layer. In [65,66], they pass the image pairs through two identical CNNs, and feed the feature vectors of the final layer to the cost module. The contrastive loss function that they use for training samples is:

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{2N} \sum_{i=1}^N (\mathbf{y}) d^{(i,L)} + (1 - \mathbf{y}) \max(m - d^{(i,L)}, 0) \quad (20)$$

where $d^{(i,L)} = \|\mathbf{z}_\alpha^{(i,L)} - \mathbf{z}_\beta^{(i,L)}\|_2^2$, and m is a margin parameter affecting non-matching pairs. If $(\mathbf{x}_\alpha^{(i)}, \mathbf{x}_\beta^{(i)})$ is a matching pair, then $\mathbf{y} = 1$. Otherwise, $\mathbf{y} = 0$.

Lin et al. [68] find that such a single margin loss function causes a dramatic drop in retrieval results when fine-tuning the

network on all pairs. Meanwhile, the performance is better retained when fine-tuning only on non-matching pairs. This indicates that the handling of matching pairs in the loss function is responsible for the drop. While the recall rate on non-matching pairs alone is stable, handling the matching pairs is the main reason for the drop in recall rate. To solve this problem, they propose a double margin loss function which adds another margin parameter to affect the matching pairs. Instead of calculating the loss of the final layer, their contrastive loss is defined for every layer l and the backpropagations for the loss of individual layers are performed at the same time. It is defined as:

$$\mathcal{L}_{d-\text{contrastive}} = \frac{1}{2N} \sum_{i=1}^N \sum_{l=1}^L (\mathbf{y}) \max(d^{(i,l)} - m_1, 0) + (1 - \mathbf{y}) \max(m_2 - d^{(i,l)}, 0) \quad (21)$$

In practice, they find that these two margin parameters can set to be equal ($m_1 = m_2 = m$) and be learned from the distribution of the sampled matching and non-matching image pairs.

3.4.4. Triplet loss

Triplet loss [69] considers three instances per loss function. The triplet units $(\mathbf{x}_a^{(i)}, \mathbf{x}_p^{(i)}, \mathbf{x}_n^{(i)})$ usually contain an anchor instance $\mathbf{x}_a^{(i)}$ as well as a positive instance $\mathbf{x}_p^{(i)}$ from the same class of $\mathbf{x}_a^{(i)}$ and a negative instance $\mathbf{x}_n^{(i)}$. Let $(\mathbf{z}_a^{(i)}, \mathbf{z}_p^{(i)}, \mathbf{z}_n^{(i)})$ denote the feature representation of the triplet units, the triplet loss is defined as:

$$\mathcal{L}_{\text{triplet}} = \frac{1}{N} \sum_{i=1}^N \max\{d_{(a,p)}^{(i)} - d_{(a,n)}^{(i)} + m, 0\} \quad (22)$$

where $d_{(a,p)}^{(i)} = \|\mathbf{z}_a^{(i)} - \mathbf{z}_p^{(i)}\|_2^2$ and $d_{(a,n)}^{(i)} = \|\mathbf{z}_a^{(i)} - \mathbf{z}_n^{(i)}\|_2^2$. The objective of triplet loss is to minimize the distance between the anchor and positive, and maximize the distance between the negative and the anchor.

However, randomly selected anchor samples may judge falsely in some special cases. For example, when $d_{(n,p)}^{(i)} < d_{(a,p)}^{(i)} < d_{(a,n)}^{(i)}$, the triplet loss may still be zero. Thus the triplet units will be neglected during the backward propagation. Liu et al. [70] propose the Coupled Clusters (CC) loss to solve this problem. Instead of using the triplet units, the coupled clusters loss function is defined over the positive set and the negative set. By replacing the randomly picked anchor with the cluster center, it makes the samples in the positive set cluster together and samples in the negative set stay relatively far away, which is more reliable than the original triplet loss. The coupled clusters loss function is defined as:

$$\mathcal{L}_{\text{cc}} = \frac{1}{N^p} \sum_{i=1}^{N^p} \frac{1}{2} \max\{\|\mathbf{z}_p^{(i)} - c_p\|_2^2 - \|\mathbf{z}_n^{(*)} - c_p\|_2^2 + m, 0\} \quad (23)$$

where N^p is the number of samples per set, $\mathbf{z}_n^{(*)}$ is the feature representation of $\mathbf{x}_n^{(*)}$ which is the nearest negative sample to the estimated center point $c_p = (\sum_i^{N^p} \mathbf{z}_p^{(i)}) / N^p$. Triplet loss and its variants have been widely used in various tasks, including re-identification [71], verification [70], and image retrieval [72].

3.4.5. Kullback-Leibler Divergence

Kullback–Leibler Divergence (KLD) is a non-symmetric measure of the difference between two probability distributions $p(x)$ and $q(x)$ over the same discrete variable x (see Fig. 7(a)). The KLD from $q(x)$ to $p(x)$ is defined as:

$$\mathcal{D}_{\text{KL}}(p||q) = -H(p(x)) - E_p[\log q(x)] \quad (24)$$

$$= \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad (25)$$

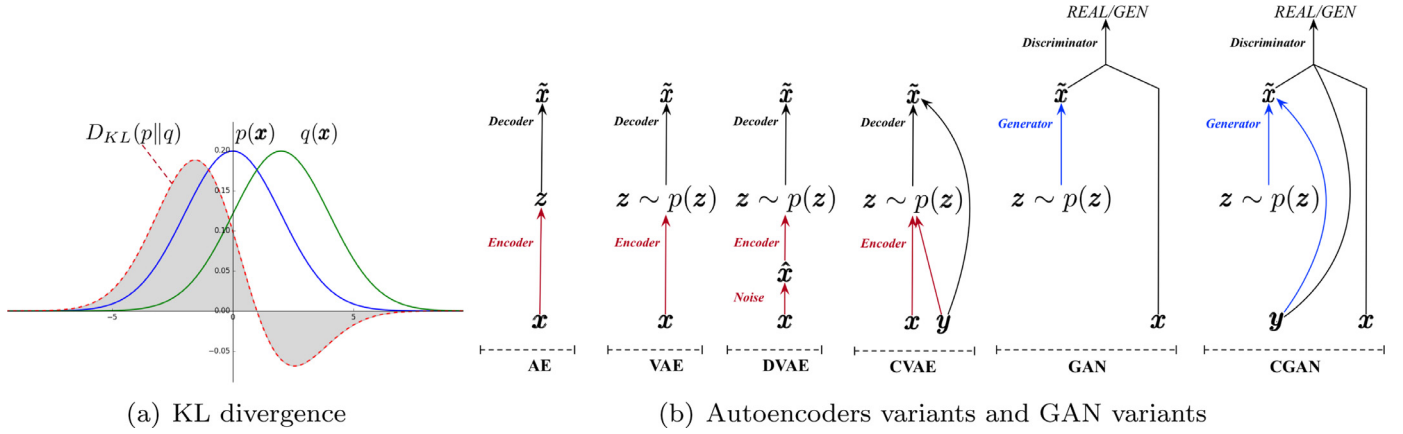


Fig. 7. The illustration of (a) the KullbackLeibler divergence for two normal Gaussian distributions, (b) AE variants (AE, VAE [73], DVAE [74], and CVAE [75]) and GAN variants (GAN [76], CGAN [77]).

where $H(p(x))$ is the Shannon entropy of $p(x)$, $E_p(\log q(x))$ is the cross entropy between $p(x)$ and $q(x)$.

KLD has been widely used as a measure of information loss in the objective function of various Autoencoders (AEs) [78–80]. Famous variants of AE include sparse AE [81,82], Denoising AE [78] and Variational AE (VAE) [73]. VAE interprets the latent representation through Bayesian inference. It consists of two parts: an encoder which “compresses” the data sample x to the latent representation $z \sim q_\phi(z|x)$; and a decoder, which maps such representation back to data space $\tilde{x} \sim p_\theta(x|z)$ which as close to the input as possible, where ϕ and θ are the parameters of encoder and decoder respectively. As proposed in [73], VAEs try to maximize the variational lower bound of the log-likelihood of $\log p(x|\phi, \theta)$:

$$\mathcal{L}_{vae} = \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - \mathcal{D}_{KL}(q_\phi(z|x) \| p(z)) \quad (26)$$

where the first term is the reconstruction cost, and the KLD term enforces prior $p(z)$ on the proposal distribution $q_\phi(z|x)$. Usually $p(z)$ is the standard normal distribution [73], discrete distribution [75], or some distributions with geometric interpretation [83]. Following the original VAE, many variants have been proposed [74,75,84]. Conditional VAE (CVAE) [75,84] generates samples from the conditional distribution with $\tilde{x} \sim p_\theta(x|y, z)$. Denoising VAE (DVAE) [74] recovers the original input x from the corrupted input $\hat{x}$ [78].

Jensen-Shannon Divergence (JSD) is a symmetrical form of KLD. It measures the similarity between $p(x)$ and $q(x)$:

$$\begin{aligned} \mathcal{D}_{JS}(p||q) &= \frac{1}{2} \mathcal{D}_{KL} \left(p(x) \left\| \frac{p(x) + q(x)}{2} \right\| \right) \\ &+ \frac{1}{2} \mathcal{D}_{KL} \left(q(x) \left\| \frac{p(x) + q(x)}{2} \right\| \right) \end{aligned} \quad (27)$$

By minimizing the JSD, we can make the two distributions $p(x)$ and $q(x)$ as close as possible. JSD has been successfully used in the Generative Adversarial Networks (GANs) [76,85,86]. In contrast to VAEs that model the relationship between x and z directly, GANs are explicitly set up to optimize for generative tasks [85]. The objective of GANs is to find the discriminator D that gives the best discrimination between the real and generated data, and simultaneously encourage the generator G to fit the real data distribution. The min-max game played between the discriminator D and the generator G is formalized by the following objective function:

$$\begin{aligned} \min_G \max_D \mathcal{L}_{gan}(D, G) &= \mathbb{E}_{x \sim p(x)} [\log D(x)] \\ &+ \mathbb{E}_{z \sim q(z)} [\log(1 - D(G(z)))] \end{aligned} \quad (28)$$

The original GAN paper [76] shows that for a fixed generator G^* , we have the optimal discriminator $D_G^*(x) = \frac{p(x)}{p(x) + q(x)}$. Then the Eq. (28) is equivalent to minimize the JSD between $p(x)$ and $q(x)$. If G and D have enough capacity, the distribution $q(x)$ converges to $p(x)$. Like Conditional VAE, the Conditional GAN (CGAN) [77] also receives an additional information y as input to generate samples conditioning on y . In practice, GANs are notoriously unstable to train [87,88].

3.5. Regularization

Overfitting is an unneglectable problem in deep CNNs, which can be effectively reduced by regularization. In the following subsection, we introduce some effective regularization techniques: ℓ_p -norm, Dropout, and DropConnect.

3.5.1. ℓ_p -norm regularization

Regularization modifies the objective function by adding additional terms that penalize the model complexity. Formally, if the loss function is $\mathcal{L}(\theta, \mathbf{x}, \mathbf{y})$, then the regularized loss will be:

$$E(\theta, \mathbf{x}, \mathbf{y}) = \mathcal{L}(\theta, \mathbf{x}, \mathbf{y}) + \lambda R(\theta) \quad (29)$$

where $R(\theta)$ is the regularization term, and λ is the regularization strength.

ℓ_p -norm regularization function is usually employed as $R(\theta) = \sum_j \|\theta_j\|_p^p$. When $p \geq 1$, the ℓ_p -norm is convex, which makes the optimization easier and renders this function attractive [17]. For $p = 2$, the ℓ_2 -norm regularization is commonly referred to as weight decay. A more principled alternative of ℓ_2 -norm regularization is Tikhonov regularization [89], which rewards invariance to noise in the inputs. When $p < 1$, the ℓ_p -norm regularization more exploits the sparsity effect of the weights but conducts to non-convex function.

3.5.2. Dropout

Dropout is first introduced by Hinton et al. [17], and it has been proven to be very effective in reducing overfitting. In [17], they apply Dropout to fully-connected layers. The output of Dropout is $\mathbf{y} = \mathbf{r} * \mathbf{a}(\mathbf{W}^T \mathbf{x})$, where $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ is the input to fully-connected layer, $\mathbf{W} \in \mathbb{R}^{n \times d}$ is a weight matrix, and $\mathbf{r}$ is a binary vector of size d whose elements are independently drawn from a Bernoulli distribution with parameter p , i.e. $r_i \sim \text{Bernoulli}(p)$. Dropout can prevent the network from becoming too dependent on any one (or any small combination) of neurons, and can force the network to be accurate even in the absence of certain information. Several methods have been proposed to improve Dropout.

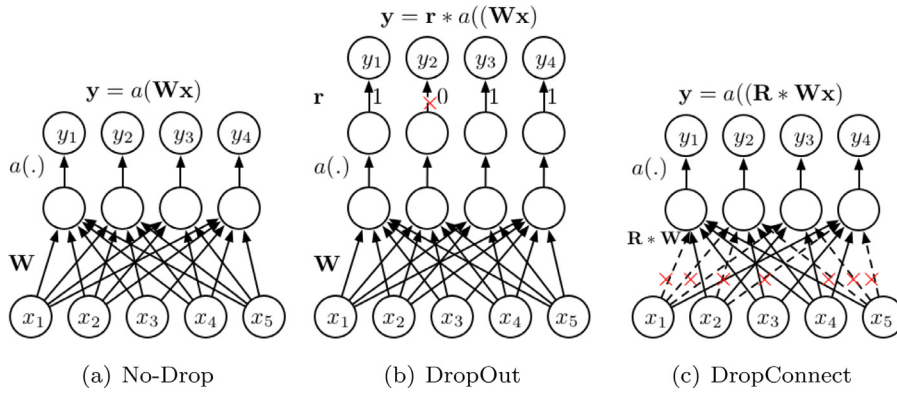


Fig. 8. The illustration of No-Drop network, DropOut network and DropConnect network.

Wang et al. [90] propose a fast Dropout method which can perform fast Dropout training by sampling from or integrating a Gaussian approximation. Ba et al. [91] propose an adaptive Dropout method, where the Dropout probability for each hidden variable is computed using a binary belief network that shares parameters with the deep network. In [92], they find that applying standard Dropout before 1×1 convolutional layer generally increases training time but does not prevent overfitting. Therefore, they propose a new Dropout method called SpatialDropout, which extends the Dropout value across the entire feature map. This new Dropout method works well especially when the training data size is small.

3.5.3. DropConnect

DropConnect [45] takes the idea of Dropout a step further. Instead of randomly setting the outputs of neurons to zero, DropConnect randomly sets the elements of weight matrix $\mathbf{W}$ to zero. The output of DropConnect is given by $\mathbf{y} = \mathbf{a}((\mathbf{R} * \mathbf{W})\mathbf{x})$, where $R_{ij} \sim \text{Bernoulli}(p)$. Additionally, the biases are also masked out during the training process. Fig. 8 illustrates the differences among No-Drop, Dropout and DropConnect networks.

3.6. Optimization

In this subsection, we discuss some key techniques for optimizing CNNs.

3.6.1. Data augmentation

Deep CNNs are particularly dependent on the availability of large quantities of training data. An elegant solution to alleviate the relative scarcity of the data compared to the number of parameters involved in CNNs is data augmentation [8]. Data augmentation consists in transforming the available data into new data without altering their natures. Popular augmentation methods include simple geometric transformations such as sampling [8], mirroring [93], rotating [94], shifting [95], and various photometric transformations [96]. Paulin et al. [97] propose a greedy strategy that selects the best transformation from a set of candidate transformations. However, their strategy involves a large number of model re-training steps, which can be computationally expensive when the number of candidate transformations is large. Hauberg et al. [98] propose an elegant way for data augmentation by randomly generating diffeomorphisms. Xie et al. [99] and Xu et al. [100] offer additional means of collecting images from the Internet to improve learning in fine-grained recognition tasks.

3.6.2. Weight initialization

Deep CNN has a huge amount of parameters and its loss function is non-convex [101], which makes it very difficult to train.

To achieve a fast convergence in training and avoid the vanishing gradient problem, a proper network initialization is one of the most important prerequisites [102,103]. The bias parameters can be initialized to zero, while the weight parameters should be initialized carefully to break the symmetry among hidden units of the same layer. If the network is not properly initialized, e.g., each layer scales its input by k , the final output will scale the original input by k^L where L is the number of layers. In this case, the value of $k > 1$ leads to extremely large values of output layers while the value of $k < 1$ leads a diminishing output value and gradients. Krizhevsky et al. [8] initialize the weights of their network from a zero-mean Gaussian distribution with standard deviation 0.01 and set the bias terms of the second, fourth and fifth convolutional layers as well as all the fully-connected layers to constant one. Another famous random initialization method is “Xavier”, which is proposed in [104]. They pick the weights from a Gaussian distribution with zero mean and a variance of $2/(n_{\text{in}} + n_{\text{out}})$, where n_{in} is the number of neurons feeding into it, and n_{out} is the number of neurons the result is fed to. Thus “Xavier” can automatically determine the scale of initialization based on the number of input and output neurons, and keep the signal in a reasonable range of values through many layers. One of its variants in Caffe² uses the n_{in} -only variant, which makes it much easier to implement. “Xavier” initialization method is later extended by He et al. [56] to account for the rectifying nonlinearities, where they derive a robust initialization method that particularly considers the ReLU nonlinearity. Their method, allows for the training of extremely deep models (e.g., [10]) to converge while the “Xavier” method [104] cannot.

Independently, Saxe et al. [105] show that orthonormal matrix initialization works much better for linear networks than Gaussian initialization, and it also works for networks with nonlinearities. Mishkin et al. [102] extend Saxe et al. [105] to an iterative procedure. Specifically, it proposes a layer-sequential unit-variance process scheme which can be viewed as an orthonormal initialization combined with batch normalization (see Section 3.6.4) performed only on the first mini-batch. It is similar to batch normalization as both of them take a unit variance normalization procedure. Differently, it uses ortho-normalization to initialize the weights which helps to efficiently de-correlate layer activities. Such an initialization technique has been applied to [106,107] with a remarkable increase in performance.

3.6.3. Stochastic gradient descent

The backpropagation algorithm is the standard training method which uses gradient descent to update the parameters. Many gradient descent optimization algorithms have been pro-

² <https://github.com/BVLC/caffe>.

posed [108,109]. Standard gradient descent algorithm updates the parameters θ of the objective $\mathcal{L}(\theta)$ as $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} E[\mathcal{L}(\theta_t)]$, where $E[\mathcal{L}(\theta_t)]$ is the expectation of $\mathcal{L}(\theta)$ over the full training set and η is the learning rate. Instead of computing $E[\mathcal{L}(\theta_t)]$, Stochastic Gradient Descent (SGD) [21] estimates the gradients on the basis of a single randomly picked example $(\mathbf{x}^{(t)}, \mathbf{y}^{(t)})$ from the training set:

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t; \mathbf{x}^{(t)}, \mathbf{y}^{(t)}) \quad (30)$$

In practice, each parameter update in SGD is computed with respect to a mini-batch as opposed to a single example. This could help to reduce the variance in the parameter update and can lead to more stable convergence. The convergence speed is controlled by the learning rate η_t . However, mini-batch SGD does not guarantee good convergence, and there are still some challenges that need to be addressed. Firstly, it is not easy to choose a proper learning rate. One common method is to use a constant learning rate that gives stable convergence in the initial stage, and then reduce the learning rate as the convergence slows down. Additionally, learning rate schedules [110,111] have been proposed to adjust the learning rate during the training. To make the current gradient update depend on historical batches and accelerate training, momentum [108] is proposed to accumulate a velocity vector in the relevant direction. The classical momentum update is given by:

$$\mathbf{v}_{t+1} = \gamma \mathbf{v}_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t; \mathbf{x}^{(t)}, \mathbf{y}^{(t)}) \quad (31)$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \quad (32)$$

where $\mathbf{v}_{t+1}$ is the current velocity vector, γ is the momentum term which is usually set to 0.9. Nesterov momentum [103] is another way of using momentum in gradient descent optimization:

$$\mathbf{v}_{t+1} = \gamma \mathbf{v}_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t + \gamma \mathbf{v}_t; \mathbf{x}^{(t)}, \mathbf{y}^{(t)}) \quad (33)$$

Compared with the classical momentum [108] which first computes the current gradient and then moves in the direction of the updated accumulated gradient, Nesterov momentum first moves in the direction of the previous accumulated gradient $\gamma \mathbf{v}_t$, calculates the gradient and then makes a gradient update. This anticipatory update prevents the optimization from moving too fast and achieves better performance [112].

Parallelized SGD methods [22,113] improve SGD to be suitable for parallel, large-scale machine learning. Unlike standard (synchronous) SGD in which the training will be delayed if one of the machines is slow, these parallelized methods use the asynchronous mechanism so that no other optimizations will be delayed except for the one on the slowest machine. Jeffrey Dean et al. [114] use another asynchronous SGD procedure called Downpour SGD to speed up the large-scale distributed training process on clusters with many CPUs. There are also some works that use asynchronous SGD with multiple GPUs. Paine et al. [115] basically combine asynchronous SGD with GPUs to accelerate the training time by several times compared to training on a single machine. Zhuang et al. [116] also use multiple GPUs to asynchronously calculate gradients and update the global model parameters, which achieves 3.2 times of speedup on 4 GPUs compared to training on a single GPU.

Note that SGD methods may not result in convergence. The training process can be terminated when the performance stops improving. A popular remedy to over-training is to use early stopping [117] in which optimization is halted based on the performance on a validation set during training. To control the duration of the training process, various stopping criteria can be considered. For example, the training might be performed for a fixed number of epochs, or until a predefined training error is reached [118]. The stopping strategy should be done carefully [119], a proper stopping strategy should let the training process continue as long as

the network generalization ability is improved and the overfitting is avoided.

3.6.4. Batch normalization

Data normalization is usually the first step of data preprocessing. Global data normalization transforms all the data to have zero-mean and unit variance. However, as the data flows through a deep network, the distribution of input to internal layers will be changed, which will lose the learning capacity and accuracy of the network. Ioffe et al. [120] propose an efficient method called Batch Normalization (BN) to partially alleviate this phenomenon. It accomplishes the so-called covariate shift problem by a normalization step that fixes the means and variances of layer inputs where the estimations of mean and variance are computed after each mini-batch rather than the entire training set. Suppose the layer to normalize has a d dimensional input, i.e., $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$. We first normalize the k th dimension as follows:

$$\hat{x}_k = (x_k - \mu_B) / \sqrt{\delta_B^2 + \epsilon} \quad (34)$$

where μ_B and δ_B^2 are the mean and variance of mini-batch respectively, and ϵ is a constant value. To enhance the representation ability, the normalized input $\hat{x}_k$ is further transformed into:

$$y_k = \text{BN}_{\gamma, \beta}(x_k) = \gamma \hat{x}_k + \beta \quad (35)$$

where γ and β are learned parameters. Batch normalization has many advantages compared with global data normalization. Firstly, it reduces internal covariant shift. Secondly, BN reduces the dependence of gradients on the scale of the parameters or of their initial values, which gives a beneficial effect on the gradient flow through the network. This enables the use of higher learning rate without the risk of divergence. Furthermore, BN regularizes the model, and thus reduces the need for Dropout. Finally, BN makes it possible to use saturating nonlinear activation functions without getting stuck in the saturated model.

3.6.5. Shortcut connections

As mentioned above, the vanishing gradient problem of deep CNNs can be alleviated by normalized initialization [8] and BN [120]. Although these methods successfully prevent deep neural networks from overfitting, they also introduce difficulties in optimizing the networks, resulting in worse performances than shallower networks [56,104,105]. Such an optimization problem suffered by deeper CNNs is regarded as the degradation problem.

Inspired by Long Short Term Memory (LSTM) networks [121] which use gate functions to determine how much of a neuron's activation value to transform or just pass through. Srivastava et al. [122] propose highway networks which enable the optimization of networks with virtually arbitrary depth. The output of their network is given by:

$$\mathbf{x}_{l+1} = \phi_{l+1}(\mathbf{x}_l, \mathbf{W}_H) \cdot \tau_{l+1}(\mathbf{x}_l, \mathbf{W}_T) + \mathbf{x}_l \cdot (\mathbf{1} - \tau_{l+1}(\mathbf{x}_l, \mathbf{W}_T)) \quad (36)$$

where $\mathbf{x}_l$ and $\mathbf{x}_{l+1}$ correspond to the input and output of l th highway block, $\tau(\cdot)$ is the transform gate and $\phi(\cdot)$ is usually an affine transformation followed by a non-linear activation function (in general it may take other forms). This gating mechanism forces the layer's inputs and outputs to be of the same size and allows highway networks with tens or hundreds of layers to be trained efficiently. The outputs of gates vary significantly with the input examples, demonstrating that the network does not just learn a fixed structure, but dynamically routes data based on specific examples.

Independently, Residual Nets (ResNets) [12] share the same core idea that works in LSTM units. Instead of employing learnable weights for neuron-specific gating, the shortcut connections in ResNets are not gated and untransformed input is directly propagated to the output which brings fewer parameters. The output of ResNets can be represented as follows:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + f_{l+1}(\mathbf{x}_l, \mathbf{W}_F) \quad (37)$$

where f_l is the weight layer, it can be a composite function of operations such as Convolution, BN, ReLU, or Pooling. With residual block, activation of any deeper unit can be written as the sum of the activation of a shallower unit and a residual function. This also implies that gradients can be directly propagated to shallower units, which makes deep ResNets much easier to be optimized than the original mapping function and more efficient to train very deep nets. This is in contrast to usual feedforward networks, where gradients are essentially a series of matrix-vector products, that may vanish as networks grow deeper.

After the original ResNets, He et al. [123] follow up with another preactivation variant of ResNets, where they conduct a set of experiments to show that identity shortcut connections are the easiest for networks to learn. They also find that bringing BN forward performs considerably better than using BN after addition. In their comparisons, the residual net with BN + ReLU pre-activation gets higher accuracies than their previous ResNets [12]. Inspired by He et al. [123], Shen et al. [124] introduce a weighting factor for the output from the convolutional layer, which gradually introduces the trainable layers. The latest Inception-v4 paper [42] also reports that training is accelerated and performance is improved by using identity skip connections across Inception modules. The original ResNets and preactivation ResNets are very deep but also very thin. By contrast, Wide ResNets [125] proposes to decrease the depth and increase the width, which achieves impressive results on CIFAR-10, CIFAR-100, and SVHN. However, their claims are not validated on the large-scale image classification task on ImageNet dataset.³ Stochastic Depth ResNets randomly drop a subset of layers and bypass them with the identity mapping for every mini-batch. By combining Stochastic Depth ResNets and Dropout, Singh et al. [126] generalize dropout and networks with stochastic depth, which can be viewed as an ensemble of ResNets, Dropout ResNets, and Stochastic Depth ResNets. The ResNets in ResNets (RiR) paper [127] describes an architecture that merges classical convolutional networks and residual networks, where each block of RiR contains residual units and non-residual blocks. The RiR can learn how many convolutional layers it should use per residual block. ResNets of ResNets (RoR) [128] is a modification to the ResNets architecture which proposes to use multi-level shortcut connections as opposed to single-level shortcut connections in the prior work on ResNets [12]. DenseNet [129] can be seen as an architecture takes the insights of the skip connection to the extreme, in which the output of a layer is connected to all the subsequent layer in the module. In all of the ResNets [12,123], Highway [122] and Inception networks [42], we can see a pretty clear trend of using shortcut connections to help train very deep networks.

4. Fast processing of CNNs

With the increasing challenges in the computer vision and machine learning tasks, the models of deep neural networks get more and more complex. These powerful models require more data for training in order to avoid overfitting. Meanwhile, the big training data also brings new challenges such as how to train the networks in a feasible amount of time. In this section, we introduce some fast processing methods of CNNs.

4.1. FFT

Mathieu et al. [49] carry out the convolutional operation in the Fourier domain with FFTs. Using FFT-based methods has many advantages. Firstly, the Fourier transformations of filters can be

reused as the filters are convolved with multiple images in a mini-batch. Secondly, the Fourier transformations of the output gradients can be reused when backpropagating gradients to both filters and input images. Finally, the summation over input channels can be performed in the Fourier domain, so that inverse Fourier transformations are only required once per output channel per image. There have already been some GPU-based libraries developed to speed up the training and testing process, such as cuDNN [130] and fbfft [131]. However, using FFT to perform convolution needs additional memory to store the feature maps in the Fourier domain, since the filters must be padded to be the same size as the inputs. This is especially costly when the striding parameter is larger than 1, which is common in many state-of-art networks, such as the early layers in [132] and [10]. While FFT can achieve faster training and testing process, the rising prominence of small size convolutional filters have become an important component in CNNs such as ResNet [12] and GoogleNet [10], which makes a new approach specialized for small filter sizes: Winograd's minimal filtering algorithms [133]. The insight of Winograd is like FFT, and the Winograd convolutions can be reduced across channels in transform space before applying the inverse transform and thus makes the inference more efficient.

4.2. Structured transforms

Low-rank matrix factorization has been exploited in a variety of contexts to improve the optimization problems. Given an $m \times n$ matrix $\mathbf{C}$ of rank r , there exists a factorization $\mathbf{C} = \mathbf{AB}$ where $\mathbf{A}$ is an $m \times r$ full column rank matrix and $\mathbf{B}$ is an $r \times n$ full row rank matrix. Thus, we can replace $\mathbf{C}$ by $\mathbf{A}$ and $\mathbf{B}$. To reduce the parameters of $\mathbf{C}$ by a fraction p , it is essential to ensure that $mr + rn < pmn$, i.e., the rank of $\mathbf{C}$ should satisfy that $r < pmn/(m+n)$. By applying this factorization, the space complexity reduces from $\mathcal{O}(mn)$ to $\mathcal{O}(r(m+n))$, and the time complexity reduces from $\mathcal{O}(mn)$ to $\mathcal{O}(r(m+n))$. To this end, Sainath et al. [134] apply the low-rank matrix factorization to the final weight layer in a deep CNN, resulting about 30–50% speedup in training time with little loss in accuracy. Similarly, Xue et al. [135] apply singular value decomposition on each layer of a deep CNN to reduce the model size by 71% with less than 1% relative accuracy loss. Inspired by [136] which demonstrates the redundancy in the parameters of deep neural networks, Denton et al. [137] and Jaderberg et al. [138] independently investigate the redundancy within the convolutional filters and develop approximations to reduced the required computations. Novikov et al. [139] generalize the low-rank ideas, where they treat the weight matrix as multi-dimensional tensor and apply a Tensor-Train decomposition [140] to reduce the number of parameters of the fully-connected layers.

Adaptive Fastfood transform is generalization of the Fastfood [141] transform for approximating matrix. It reparameterizes the weight matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$ in fully-connected layers with an Adaptive Fastfood transformation: $\mathbf{Cx} = (\tilde{\mathbf{D}}_1 \mathbf{H} \tilde{\mathbf{D}}_2 \Pi \mathbf{H} \tilde{\mathbf{D}}_3) \mathbf{x}$, where $\tilde{\mathbf{D}}_1$, $\tilde{\mathbf{D}}_2$ and $\tilde{\mathbf{D}}_3$ are diagonal matrices of parameters, Π is a random permutation matrix, and $\mathbf{H}$ denotes the Walsh–Hadamard matrix. The space complexity of Adaptive Fastfood transform is $\mathcal{O}(n)$, and the time complexity is $\mathcal{O}(n \log n)$.

Motivated by the great advantages of circulant matrix in both space and computation efficiency [142,143], Cheng et al. [144] explore the redundancy in the parametrization of fully-connected layers by imposing the circulant structure on the weight matrix to speed up the computation, and further allow the use of FFT for faster computation. With a circulant matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$ as the matrix of parameters in a fully-connected layer, for an input vector $\mathbf{x} \in \mathbb{R}^n$, the output of $\mathbf{Cx}$ can be calculated efficiently using the FFT and inverse IFFT: $\mathbf{CDx} = \text{ifft}(\text{fft}(\mathbf{v})) \circ \text{fft}(\mathbf{x})$, where $\circ$ corresponds to elementwise multiplication operation, $\mathbf{v} \in \mathbb{R}^n$ is defined by $\mathbf{C}$, and

³ <http://www.image-net.org>.

$\mathbf{D}$ is a random sign flipping matrix for improving the capacity of the model. This method reduces the time complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$, and space complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$. Moczulski et al. [145] further generalize the circulant structures by interleaving diagonal matrices with the orthogonal Discrete Cosine Transform (DCT). The resulting transform, $\mathbf{ACDC}^{-1}$, has $\mathcal{O}(n)$ space complexity and $\mathcal{O}(n \log n)$ time complexity.

4.3. Low precision

Floating point numbers are a natural choice for handling the small updates of the parameters of CNNs. However, the resulting parameters may contain a lot of redundant information [146]. To reduce redundancy, Binarized Neural Networks (BNNs) restrict some or all the arithmetics involved in computing the outputs to be binary values.

There are three aspects of binarization for neural network layers: binary input activations, binary synapse weights, and binary output activations. Full binarization requires all the three components are binarized, and the cases with one or two components are considered as partial binarization. Kim et al. [147] consider full binarization with a predetermined portion of the synapses having zero weight, and all other synapses with a weight of one. Their network only needs XNOR and bit count operations, and they report 98.7% accuracy on the MNIST dataset. XNOR-Net [148] applies convolutional BNNs on the ImageNet dataset with topologies inspired by AlexNet, ResNet and GoogLeNet, reporting top-1 accuracies of up to 51.2% for full binarization and 65.5% for partial binarization. DoReFa-Net [149] explores reducing precision during the forward pass as well as the backward pass. Both partial and full binarization are explored in their experiments and the corresponding top-1 accuracies on ImageNet are 43% and 53%. The work by Courbariaux et al. [150] describes how to train fully-connected networks and CNNs with full binarization and batch normalization layers, reporting competitive accuracies on the MNIST, SVHN, and CIFAR-10 datasets.

4.4. Weight compression

Many attempts have been made to reduce the number of parameters in the convolution layers and fully-connected layers. Here, we briefly introduce some methods under these topics: vector quantization, pruning, and hashing.

Vector Quantization (VQ) is a method for compressing densely connected layers to make CNN models smaller. Similar to scalar quantization where a large set of numbers is mapped to a smaller set [151], VQ quantizes groups of numbers together rather than addressing them one at a time. In 2013, Denil et al. [136] demonstrate the presence of redundancy in neural network parameters, and use VQ to significantly reduce the number of dynamic parameters in deep models. Gong et al. [152] investigate the information theoretical vector quantization methods for compressing the parameters of CNNs, and they obtain parameter prediction results similar to those of [136]. They also find that VQ methods have a clear gain over existing matrix factorization methods, and among the VQ methods, structured quantization methods such as product quantization work significantly better than other methods (e.g., residual quantization [153], scalar quantization [154]).

An alternative approach to weight compression is pruning. It reduces the number of parameters and operations in CNNs by permanently dropping less important connections [155], which enables smaller networks to inherit knowledge from the large predecessor networks and maintains comparable performance. Han et al. [146,156] introduce fine-grained sparsity in a network by a magnitude-based pruning approach. If the absolute magnitude of any weight is less than a scalar threshold, the weight

is pruned. Gao et al. [157] extend the magnitude-based approach to allow restoration of the pruned weights in the previous iterations, with tightly coupled pruning and retraining stages, for greater model compression. Yang et al. [158] take the correlation between weights into consideration and propose an energy-aware pruning algorithm that directly uses energy consumption estimation of a CNN to guide the pruning process. Rather than fine-grained pruning, there are also works that investigate coarse-grained pruning. Hu et al. [159] propose removing filters that frequently generate zero output activations on the validation set. Srinivas et al. [160] merge similar filters into one, while Mariet et al. [161] merge filters with similar output activations into one.

Designing a proper hashing technique to accelerate the training of CNNs or save memory space also an interesting problem. HashedNets [162] is a recent technique to reduce model sizes by using a hash function to group connection weights into hash buckets, and all connections within the same hash bucket share a single parameter value. Their network shrinks the storage costs of neural networks significantly while mostly preserves the generalization performance in image classification. As pointed out in Shi et al. [163] and Weinberger et al. [164], sparsity will minimize hash collision making feature hashing even more effective. HashNets may be used together with pruning to give even better parameter savings.

4.5. Sparse convolution

Recently, several attempts have been made to sparsify the weights of convolutional layers [165,166]. Liu et al. [165] consider sparse representations of the basis filters, and achieve 90% sparsifying by exploiting both inter-channel and intra-channel redundancy of convolutional kernels. Instead of sparsifying the weights of convolution layers, Wen et al. [166] propose a Structured Sparsity Learning (SSL) approach to simultaneously optimize their hyperparameters (filter size, depth, and local connectivity). Bagherinezhad et al. [167] propose a lookup-based convolutional neural network (LCNN) that encodes convolutions by few lookups to a rich set of dictionary that is trained to cover the space of weights in CNNs. They decode the weights of the convolutional layer with a dictionary and two tensors. The dictionary is shared among all weight filters in a layer, which allows a CNN to learn from very few training examples. LCNN can achieve a higher accuracy in a small number of iterations compared to standard CNN.

5. Applications of CNNs

In this section, we introduce some recent works that apply CNNs to achieve state-of-the-art performance, including image classification, object tracking, pose estimation, text detection, visual saliency detection, action recognition, scene labeling, speech and natural language processing.

5.1. Image classification

CNNs have been applied in image classification for a long time [168–171]. Compared with other methods, CNNs can achieve better classification accuracy on large scale datasets [8,9,172] due to their capability of joint feature and classifier learning. The breakthrough of large scale image classification comes in 2012. Krizhevsky et al. [8] develop the AlexNet and achieve the best performance in ILSVRC 2012. After the success of AlexNet, several works have made significant improvements in classification accuracy by either reducing filter size [11] or expanding the network depth [9,10].

Building a hierarchy of classifiers is a common strategy for image classification with a large number of classes [173]. The work of Srivastava and Salakhutdinov [174] is one of the earliest attempts to introduce category hierarchy in CNN, in which a discriminative transfer learning with tree-based priors is proposed. They use a hierarchy of classes for sharing information among related classes in order to improve performance for classes with very few training examples. Similarly, Wang et al. [175] build a tree structure to learn fine-grained features for subcategory recognition. Xiao et al. [176] propose a training method that grows a network not only incrementally but also hierarchically. In their method, classes are grouped according to similarities and are self-organized into different levels. Yan et al. [177] introduce a hierarchical deep CNNs (HD-CNNs) by embedding deep CNNs into a category hierarchy. They decompose the classification task into two steps. The coarse category CNN classifier is first used to separate easy classes from each other, and then those more challenging classes are routed downstream to fine category classifiers for further prediction. This architecture follows the coarse-to-fine classification paradigm and can achieve lower error at the cost of an affordable increase of complexity.

Subcategory classification is another rapidly growing subfield of image classification. There are already some fine-grained image datasets (such as Birds [178], Dogs [179], Cars [180], and Plants [181]). Using object part information is beneficial for fine-grained classification [182]. Generally, the accuracy can be improved by localizing important parts of objects and representing their appearances discriminatively. Along this way, Branson et al. [183] propose a method which detects parts and extracts CNN features from multiple pose-normalized regions. Part annotation information is used to learn a compact pose normalization space. They also build a model that integrates lower-level feature layers with pose-normalized extraction routines and higher-level feature layers with unaligned image features to improve the classification accuracy. Zhang et al. [184] propose a part-based R-CNN which can learn whole-object and part detectors. They use selective search [185] to generate the part proposals, and apply non-parametric geometric constraints to more accurately localize parts. Lin et al. [186] incorporate part localization, alignment, and classification into one recognition system which is called Deep LAC. Their system is composed of three sub-networks: localization sub-network is used to estimate the part location, alignment sub-network receives the location as input and performs template alignment [187], and classification sub-network takes pose aligned part images as input to predict the category label. They also propose a value linkage function to link the sub-networks and make them work as a whole in training and testing.

As can be noted, all the above-mentioned methods make use of part annotation information for supervised training. However, these annotations are not easy to collect and these systems have difficulty in scaling up and to handle many types of fine-grained classes. To avoid this problem, some researchers propose to find localized parts or regions in an unsupervised manner. Krause et al. [188] use the ensemble of localized learned feature representations for fine-grained classification, they use co-segmentation and alignment to generate parts, and then compare the appearance of each part and aggregate the similarities together. In their latest paper [189], they combine co-segmentation and alignment in a discriminative mixture to generate parts for facilitating fine-grained classification. Zhang et al. [190] use the unsupervised selective search to generate object proposals, and then select the useful parts from the multi-scale generated part proposals. Xiao et al. [191] apply visual attention in CNN for fine-grained classification. Their classification pipeline is composed of three types of attentions: the bottom-up attention proposes candidate patches, the object-level top-down attention selects relevant

patches of a certain object, and the part-level top-down attention localizes discriminative parts. These attentions are combined to train domain-specific networks which can help to find foreground object or object parts and extract discriminative features. Lin et al. [192] propose a bilinear model for fine-grained image classification. The recognition architecture consists of two feature extractors. The outputs of two feature extractors are multiplied using the outer product at each location of the image, and are pooled to obtain an image descriptor.

5.2. Object detection

Object detection has been a long-standing and important problem in computer vision [193–195]. Generally, the difficulties mainly lie in how to accurately and efficiently localize objects in images or video frames. The use of CNNs for detection and localization can be traced back to 1990s [196]. However, due to the lack of training data and limited processing resources, the progress of CNN-based object detection is slow before 2012. Since 2012, the huge success of CNNs in ImageNet challenge [8] rekindles interest in CNN-based object detection [197]. In some early works [196,198], they use the sliding window based approaches to densely evaluate the CNN classifier on windows sampled at each location and scale. Since there are usually hundreds of thousands of candidate windows in a image, these methods suffer from highly computational cost, which makes them unsuitable to be applied on the large-scale dataset, e.g., Pascal VOC [172], ImageNet [8] and MSCOCO [199].

Recently, object proposal based methods attract a lot of interests and are widely studied in the literature [185,193,200,201]. These methods usually exploit fast and generic measurements to test whether a sampled window is a potential object or not, and further pass the output object proposals to more sophisticated detectors to determine whether they are background or belong to a specific object class. One of the most famous object proposal based CNN detector is Region-based CNN (R-CNN) [202]. R-CNN uses Selective Search (SS) [185] to extract around 2000 bottom-up region proposals that are likely to contain objects. Then, these region proposals are warped to a fixed size (227×227), and a pre-trained CNN is used to extract features from them. Finally, a binary SVM classifier is used for detection.

R-CNN yields a significant performance boost. However, its computational cost is still high since the time-consuming CNN feature extractor will be performed for each region separately. To deal with this problem, some recent works propose to share the computation in feature extraction [9,28,132,202]. OverFeat Sermanet et al. [132] computes CNN features from an image pyramid for localization and detection. Hence the computation can be easily shared between overlapping windows. Spatial pyramid pooling network (SPP net) [203] is a pyramid-based version of R-CNN [202], which introduces an SPP layer to relax the constraint that input images must have a fixed size. Unlike R-CNN, SPP net extracts the feature maps from the entire image only once, and then applies spatial pyramid pooling on each candidate window to get a fixed-length representation. A drawback of SPP net is that its training procedure is a multi-stage pipeline, which makes it impossible to train the CNN feature extractor and SVM classifier jointly to further improve the accuracy. Fast RCNN [204] improves SPP net by using an end-to-end training method. All network layers can be updated during fine-tuning, which simplifies the learning process and improves detection accuracy. Later, Faster R-CNN [204] introduces a region proposal network (RPN) for object proposals generation and achieves further speed-up. Beside R-CNN based methods, Gidaris et al. [205] propose a multi-region and semantic segmentation-aware model for object detection. They integrate the combined features on an iterative localization mechanism as well as a box-voting scheme after non-max suppression.

Yoo et al. [206] treat the object detection problem as an iterative classification problem. It predicts an accurate object boundary box by aggregating quantized weak directions from their detection network.

Another important issue of object detection is how to explore effective training sets as the performance is somehow largely depends on quantity and quality of both positive and negative samples. Online bootstrapping (or hard negative mining [207]) for CNN training has recently gained interest due to its importance for intelligent cognitive systems interacting with dynamically changing environments [208]. Shrivastava et al. [209] proposes a novel bootstrapping technique called online hard example mining (OHEM) for training detection models based on CNNs. It simplifies the training process by automatically selecting the hard examples. Meanwhile, it only computes the feature maps of an image once, and then forwards all region-of-interests (RoIs) of the image on top of these feature maps. Thus it is able to find the hard examples with a small extra computational cost.

More recently, YOLO [210] and SSD [211] allow single pipeline detection that directly predicts class labels. YOLO [210] treats object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. The whole detection pipeline is a single network which predicts bounding boxes and class probabilities from the full image in one evaluation, and can be optimized end-to-end directly on detection performance. SSD [211] discretizes the output space of bounding boxes into a set of default boxes over different aspect ratios and scales per feature map location. With this multiple scales setting and their matching strategy, SSD is significantly more accurate than YOLO. With the benefits from super-resolution, Lu et al. [212] propose a top-down search strategy to divide a window into sub-windows recursively, in which an additional network is trained to account for such division decisions.

5.3. Object tracking

The success in object tracking relies heavily on how robust the representation of target appearance is against several challenges such as view point changes, illumination changes, and occlusions [213–215]. There are several attempts to employ CNNs for visual tracking. Fan et al. [216] use CNN as a base learner. It learns a separate class-specific network to track objects. In [216], the authors design a CNN tracker with a shift-variant architecture. Such an architecture plays a key role so that it turns the CNN model from a detector into a tracker. The features are learned during off-line training. Different from traditional trackers which only extract local spatial structures, this CNN based tracking method extracts both spatial and temporal structures by considering the images of two consecutive frames. Because the large signals in the temporal information tend to occur near objects that are moving, the temporal structures provide a crude velocity signal to tracking.

Li et al. [217] propose a target-specific CNN for object tracking, where the CNN is trained incrementally during tracking with new examples obtained online. They employ a candidate pool of multiple CNNs as a data-driven model of different instances of the target object. Individually, each CNN maintains a specific set of kernels that favourably discriminate object patches from their surrounding background using all available low-level cues. These kernels are updated in an online manner at each frame after being trained with just one instance at the initialization of the corresponding CNN. Instead of learning one complicated and powerful CNN model for all the appearance observations in the past, Li et al. [217] use a relatively small number of filters in the CNN within a framework equipped with a temporal adaptation mechanism. Given a frame, the most promising CNNs in the pool are selected to evaluate the hypotheses for the target object. The hypothesis with the highest

score is assigned as the current detection window and the selected models are retrained using a warm-start backpropagation which optimizes a structural loss function.

In [218], a CNN object tracking method is proposed to address limitations of handcrafted features and shallow classifier structures in object tracking problem. The discriminative features are first automatically learned via a CNN. To alleviate the tracker drifting problem caused by model update, the tracker exploits the ground truth appearance information of the object labeled in the initial frames and the image observations obtained online. A heuristic schema is used to judge whether updating the object appearance models or not.

Hong et al. [219] propose a visual tracking algorithm based on a pre-trained CNN, where the network is trained originally for large-scale image classification and the learned representation is transferred to describe target. On top of the hidden layers in the CNN, they put an additional layer of an online SVM to learn a target appearance discriminatively against background. The model learned by SVM is used to compute a target-specific saliency map by back-projecting the information relevant to target to input image space. And they exploit the target-specific saliency map to obtain generative target appearance models and perform tracking with understanding of spatial configuration of target.

5.4. Pose estimation

Since the breakthrough in deep structure learning, many recent works pay more attention to learn multiple levels of representations and abstractions for human-body pose estimation task with CNNs [220,221]. DeepPose [222] is the first application of CNNs to human pose estimation problem. In this work, pose estimation is formulated as a CNN-based regression problem to body joint coordinates. A cascade of 7-layered CNNs are presented to reason about pose in a holistic manner. Unlike the previous works that usually explicitly design graphical model and part detectors, the DeepPose captures the full context of each body joint by taking the whole image as the input.

Meanwhile, some works exploit CNN to learn representation of local body parts. Ajrun et al. [223] present a CNN based end-to-end learning approach for full-body human pose estimation, in which CNN part detectors and an Markov Random Field (MRF)-like spatial model are jointly trained, and pair-wise potentials in the graph are computed using convolutional priors. In a series of papers, Tompson et al. [224] use a multi-resolution CNN to compute heat-map for each body part. Different from [223], Tompson et al. [224] learn the body part prior model and implicitly the structure of the spatial model. Specifically, they start by connecting every body part to itself and to every other body part in a pair-wise fashion, and use a fully-connected graph to model the spatial prior. As an extension of [224], Tompson et al. [92] propose a CNN architecture which includes a position refinement model after a rough pose estimation CNN. This refinement model, which is a Siamese network [64], is jointly trained in cascade with the off-the-shelf model [224]. In a similar work with [224], Chen et al. [225,226] also combine graphical model with CNN. They exploit a CNN to learn conditional probabilities for the presence of parts and their spatial relationships, which are used in unary and pairwise terms of the graphical model. The learned conditional probabilities can be regarded as low-dimensional representations of the body pose. There is also a pose estimation method called dual-source CNN [227] that integrates graphical models and holistic style. It takes the full body image and the holistic view of the local parts as inputs to combine both local and contextual information.

In addition to still image pose estimation with CNN, recently researchers also apply CNN to human pose estimation in videos. Based on the work [224], Jain et al. [228] also incorporate RGB

features and motion features to a multi-resolution CNN architecture to further improve accuracy. Specifically, The CNN works in a sliding-window manner to perform pose estimation. The input of the CNN is a 3D tensor which consists of an RGB image and its corresponding motion features, and the output is a 3D tensor containing response-maps of the joints. In each response map, the value of each location denote the energy for presence the corresponding joint at that pixel location. The multi-resolution processing is achieved by simply down sampling the inputs and feeding them to the network.

5.5. Text detection and recognition

The task of recognizing text in image has been widely studied for a long time [229–232]. Traditionally, optical character recognition (OCR) is the major focus. OCR techniques mainly perform text recognition on images in rather constrained visual environments (e.g., clean background, well-aligned text). Recently, the focus has been shifted to text recognition on scene images due to the growing trend of high-level visual understanding in computer vision research [233,234]. The scene images are captured in unconstrained environments where there exists a large amount of appearance variations which poses great difficulties to existing OCR techniques. Such a concern can be mitigated by using stronger and richer feature representations such as those learned by CNN models. Along the line of improving the performance of scene text recognition with CNN, a few works have been proposed. The works can be coarsely categorized into three types: (1) text detection and localization without recognition, (2) text recognition on cropped text images, and (3) end-to-end text spotting that integrates both text detection and recognition:

5.5.1. Text detection

One of the pioneering works to apply CNN for scene text detection is [235]. The CNN model employed by Delakis and Garcia [235] learns on cropped text patches and non-text scene patches to discriminate between the two. The text are then detected on the response maps generated by the CNN filters given the multiscale image pyramid of the input. To reduce the search space for text detection, Xu et al. [236] propose to obtain a set of character candidates via Maximally Stable Extremal Regions (MSER) and filter the candidates by CNN classification. Another work that combines MSER and CNN for text detection is [237]. In [237], CNN is used to distinguish text-like MSER components from non-text components, and cluttered text components are split by applying CNN in a sliding window manner followed by Non-Maximal Suppression (NMS). Other than localization of text, there is an interesting work [238] that makes use of CNN to determine whether the input image contains text, without telling where the text is exactly located. In [238], text candidates are obtained using MSER which are then passed into a CNN to generate visual features, and lastly the global features of the images are constructed by aggregating the CNN features in a Bag-of-Words (BoW) framework.

5.5.2. Text recognition

Goodfellow et al. [239] propose a CNN model with multiple softmax classifiers in its final layer, which is formulated in such a way that each classifier is responsible for character prediction at each sequential location in the multi-digit input image. As an attempt to recognize text without using lexicon and dictionary, Jaderberg et al. [240] introduce a novel Conditional Random Fields (CRF)-like CNN model to jointly learn character sequence prediction and bigram generation for scene text recognition. The more recent text recognition methods supplement conventional CNN models with variants of recurrent neural networks (RNN) to better model the sequence dependencies between characters in text. In

[241], CNN extracts rich visual features from character-level image patches obtained via sliding window, and the sequence labelling is carried out by LSTM [242]. The method presented in [243] is very similar to [241], except that in [243], lexicon can be taken into consideration to enhance text recognition performance.

5.5.3. End-to-end text spotting

For end-to-end text spotting, Wang et al. [15] apply a CNN model originally trained for character classification to perform text detection. Going in a similar direction as [15], the CNN model proposed in [244] enables feature sharing across the four different subtasks of an end-to-end text spotting system: text detection, character case-sensitive and insensitive classification, and bigram classification. Jaderberg et al. [245] make use of CNNs in a very comprehensive way to perform end-to-end text spotting. In [245], the major subtasks of its proposed system, namely text bounding box filtering, text bounding box regression, and text recognition are each tackled by a separate CNN model.

5.6. Visual saliency detection

The technique to locate important regions in imagery is referred to as visual saliency prediction. It is a challenging research topic, with a vast number of computer vision and image processing applications facilitated by it. Recently, a couple of works have been proposed to harness the strong visual modeling capability of CNNs for visual saliency prediction.

Multi-contextual information is a crucial prior in visual saliency prediction, and it has been used concurrently with CNN in most of the considered works [246–250]. Wang et al. [246] introduce a novel saliency detection algorithm which sequentially exploits local context and global context. The local context is handled by a CNN model which assigns a local saliency value to each pixel given the input of local image patches, while the global context (object-level information) is handled by a deep fully-connected feedforward network. In [247], the CNN parameters are shared between the global-context and local-context models, for predicting the saliency of superpixels found within object proposals. The CNN model adopted in [248] is pre-trained on large-scale image classification dataset and then shared among different contextual levels for feature extraction. The outputs of the CNN at different contextual levels are then concatenated as input to be passed into a trainable fully-connected feedforward network for saliency prediction. Similar to [247,248], the CNN model used in [249] for saliency prediction are shared across three CNN streams, with each stream taking input of a different contextual scale. He et al. [250] derive a spatial kernel and a range kernel to produce two meaningful sequences as 1-D CNN inputs, to describe color uniqueness and color distribution respectively. The proposed sequences are advantageous over inputs of raw image pixels because they can reduce the training complexity of CNN, while being able to encode the contextual information among superpixels.

There are also CNN-based saliency prediction approaches [251–253] that do not consider multi-contextual information. Instead, they rely very much on the powerful representation capability of CNN. In [251], an ensemble of CNNs is derived from a large number of randomly instantiated CNN models, to generate good features for saliency detection. The CNN models instantiated in [251] are however not deep enough because the maximum number of layers is capped at three. By using a pre-trained and deeper CNN model with 5 convolutional layers, Kümmerer et al. [252] (Deep Gaze) learns a separate saliency model to jointly combine the responses from every CNN layer and predict saliency values. Pan and Gir-i Nieto [253] is the only work making use of CNN to perform visual saliency prediction in an end-to-end manner, which means the CNN model accepts raw pixels as input and produces saliency

map as output. Pan et al. [253] argue that the success of the proposed end-to-end method is attributed to its not-so-deep CNN architecture which attempts to prevent overfitting.

5.7. Action recognition

Action recognition, the behaviour analysis of human subjects and classifying their activities based on their visual appearance and motion dynamics, is one of the challenging problems in computer vision [254–256]. Generally, this problem can be divided to two major groups: action analysis in still images and in videos. For both of these two groups, effective CNN based methods have been proposed. In this subsection we briefly introduce the latest advances on these two groups.

5.7.1. Action recognition in still images

The work of Donahue et al. [257] has shown the output of last few layers of a trained CNN can be used as a general visual feature descriptor for a variety of tasks. The same intuition is utilized for action recognition by Simonyan and Zisserman [9] and Oquab et al. [258], in which they use the outputs of the penultimate layer of a pre-trained CNN to represent full images of actions as well as the human bounding boxes inside them, and achieve a high level of performance in action classification. Gkioxari et al. [259] add a part detection to this framework. Their part detector is a CNN based extension to the original Poselet [260] method.

CNN based representation of contextual information is utilized for action recognition in [261]. They search for the most representative secondary region within a large number of object proposal regions in the image and add contextual features to the description of the primary region (ground truth bounding box of human subject) in a bottom-up manner. They utilize a CNN to represent and fine-tune the representations of the primary and the contextual regions. After that, they move a step forward and show that it is possible to locate and recognize human actions in images without using human bounding boxes [262]. However, they need to train human detectors to guide their recognition at test time. In [263], they propose a method that segments out the action mask of underlying human-object interactions with minimum annotation efforts.

5.7.2. Action Recognition in Video Sequences

Applying CNNs on videos is challenging because traditional CNNs are designed to represent two dimensional pure spatial signals but in videos a new temporal axis is added which is essentially different from the spatial variations in images [256,264]. The sizes of the video signals are also in higher orders in comparison to those of images which makes it more difficult to apply convolutional networks on. Ji et al. [265] propose to consider the temporal axis in a similar manner as other spatial axes and introduce a network of 3D convolutional layers to be applied on video inputs. Recently Tran et al. [266] study the performance, efficiency, and effectiveness of this approach and show its strengths compared to other approaches.

Another approach to apply CNNs on videos is to keep the convolutions in 2D and fuse the feature maps of consecutive frames, as proposed by Karpathy et al. [267]. They evaluate three different fusion policies: late fusion, early fusion, and slow fusion, and compare them with applying the CNN on individual single frames. One more step forward for better action recognition via CNNs is to separate the representation to spatial and temporal variations and train individual CNNs for each of them, as proposed by Simonyan and Zisserman [268]. First stream of this framework is a traditional CNN applied on all the frames and the second receives the dense optical flow of the input videos and trains another CNN which is identical to the spatial stream in size and structure. The output of

the two streams are combined in a class score fusion step. Chéron et al. [269] utilize the two stream CNN on the localized parts of the human body and show the aggregation of part-based local CNN descriptors can effectively improve the performance of action recognition. Another approach to model the dynamics of videos differently from spatial variations, is to feed the CNN based features of individual frames to a sequence learning module e.g., a recurrent neural network. Donahue et al. [270] study different configurations of applying LSTM units as the sequence learner in this framework.

5.8. Scene labeling

Scene labeling aims to relate one semantic class (road, water, sea etc.) to each pixel of the input image [271–275]. CNNs are used to model the class likelihood of pixels directly from local image patches. They are able to learn strong features and classifiers to discriminate the local visual subtleties. Farabet et al. [276] have pioneered to apply CNNs to scene labeling tasks. They feed their Multi-scale ConvNet with different scale image patches, and they show that the learned network is able to perform much better than systems with hand-crafted features. Besides, this network is also successfully applied to RGB-D scene labeling [277]. To enable the CNNs to have a large field of view over pixels, Pinheiro et al. [278] develop the recurrent CNNs. More specifically, the identical CNNs are applied recurrently to the output maps of CNNs in the previous iterations. By doing this, they can achieve slightly better labeling results while significantly reduces the inference times. Shuai et al. [279–281] train the parametric CNNs by sampling image patches, which speeds up the training time dramatically. They find that patch-based CNNs suffer from local ambiguity problems, and Shuai et al. [279] solve it by integrating global beliefs. Shuai et al. [280] and Shuai et al. [281] use the recurrent neural networks to model the contextual dependencies among image features from CNNs, and dramatically boost the labeling performance.

Meanwhile, researchers are exploiting to use the pre-trained deep CNNs for object semantic segmentation. Mostajabi et al. [282] apply the local and proximal features from a ConvNet and apply the Alex-net [8] to obtain the distant and global features, and their concatenation gives rise to the zoom-out features. They achieve very competitive results on the semantic segmentation tasks. Long et al. [28] train a fully convolutional Network to directly predict the input images to dense label maps. The convolution layers of the FCNs are initialized from the model pre-trained on ImageNet classification dataset, and the deconvolution layers are learned to upsample the resolution of label maps. Chen et al. [283] also apply the pre-trained deep CNNs to emit the labels of pixels. Considering that the imperfectness of boundary alignment, they further use fully connected CRF to boost the labeling performance.

5.9. Speech processing

5.9.1. Automatic speech recognition

Automatic Speech Recognition (ASR) is the technology that converts human speech into spoken words [284]. Before applying CNN to ASR, this domain has long been dominated by the Hidden Markov Model and Gaussian Mixture Model (GMM-HMM) methods [285], which usually require extracting hand-craft features on speech signals, e.g., the most popular Mel Frequency Cepstral Coefficients (MFCC) features. Meanwhile, some researchers have applied Deep Neural Networks (DNNs) in large vocabulary continuous speech recognition (LVCSR) and obtained encouraging results [286,287], however, their networks are susceptible to performance degradations under mismatched condition [288], such as different recording conditions etc.

CNNs have shown better performance over GMM-HMMs and general DNNs [289,290], since they are well suited to exploit the correlations in both time and frequency domains through the local connectivity and are capable of capturing frequency shift in human speech signals. In [289], they achieve lower speech recognition errors by applying CNN on Mel filter bank features. Some attempts use the raw waveform with CNNs, and to learn filters to process the raw waveform jointly with the rest of the network [291,292]. Most of the early applications of CNN in ASR only use fewer convolution layers. For example, Abdel-Hamid et al. [290] use one convolutional layer in their network, and Amodei et al. [293] use three convolutional layers as the feature preprocessing layer. Recently, very deep CNNs have shown impressive performance in ASR [294,295]. Besides, small filters have successfully applied in acoustic modeling in hybrid NN-HMM ASR system, and pooling operations are replaced by densely connected layers for ASR tasks [296]. Yu et al. [297] propose a layer-wise context expansion with attention model for ASR. It is a variant of time-delay neural network [298] in which lower layers focus on extracting simple local patterns while higher layers exploit broader context and extract complex patterns than the lower layers. A similar idea can be found in [40].

5.9.2. Statistical Parametric Speech Synthesis

In addition to speech recognition, the impact of CNNs has also spread to Statistical Parametric Speech Synthesis (SPSS). The goal of speech synthesis is to generate speech sounds directly from the text and possibly with additional information. It has been known for many years that the speech sounds generated by shallow structured HMM networks are often muffled compared with natural speech. Many studies have adopted deep learning to overcome such deficiency [299–301]. One advantage of these methods is their strong ability to represent the intrinsic correlation by using a generative modeling framework. Inspired by the recent advances in neural autoregressive generative models that model complex distributions such as images [302] and text [303], WaveNet [39] makes use of the generative model of the CNN to represent the conditional distribution of the acoustic features given the linguistic features, which can be seen as a milestone in SPSS. In order to deal with long-range temporal dependencies, they develop a new architecture based on dilated causal convolutions to capture very large receptive fields. By conditioning linguistic features on text, it can be used to directly synthesize speech from text.

5.10. Natural language processing

5.10.1. Statistical language modeling

For statistical language modeling, the input typically consists of incomplete sequences of words rather than complete sentences [304,305]. Kim et al. [304] use the output of character-level CNN as the input to an LSTM at each time step. *genCNN* [306] is a convolutional architecture for sequence prediction, which uses separate gating networks to replace the max-pooling operations. Recently, Kalchbrenner et al. [38] propose a CNN-based architecture for sequence processing called ByteNet, which is a stack of two dilated CNNs. Like WaveNet [39], ByteNet also benefits from convolutions with dilations to increase the receptive field size, thus can model sequential data with long-term dependencies. It also has the advantage that the computational time only linearly depends on the length of the sequences. Compared with recurrent neural networks, CNNs not only can get long-range information but also get a hierarchical representation of the input words. Gu et al. [307] and Yann et al. [308] share a similar idea that both of them use CNN without pooling to model the input words. Gu et al. [307] combine the language CNN with recurrent highway

networks and achieve a huge improvement compared to LSTM-based methods. Inspired by the gating mechanism in LSTM networks, the gated CNN in [308] uses a gating mechanism to control the path through which information flows in the network, and achieves the state-of-the-art on WikiText-103. However, the frameworks in [308] and [307] are still under the recurrent framework, and the input window size of their network are of limited size. How to capture the specific long-term dependencies as well as hierarchical representation of history words is still an open problem.

5.10.2. Text classification

Text classification is a crucial task for Natural Language Processing (NLP). Natural language sentences have complicated structures, both sequential and hierarchical, that are essential for understanding them. Owing to the powerful capability of capturing local relations of temporal or hierarchical structures, CNNs have achieved top performance in sentence modeling. A proper CNN architecture is important for text classification. Collobert et al. [309] and Yu et al. [310] apply one convolutional layer to model the sentence, while Kalchbrenner et al. [311] stack multiple layers of convolution to model sentences. In [312], they use multichannel convolution and variable kernels for sentence classification. It is shown that multiple convolutional layers help to extract high-level abstract features, and multiple linear filters can effectively consider different n -gram features. Recently, Yin et al. [313] extend the network in [312] by hierarchical convolution architecture and further exploration of multichannel and variable size feature detectors. The pooling operation can help the network deal with variable sentence lengths. In [312,314], they use max-pooling to keep the most important information to represent the sentence. However, max-pooling cannot distinguish whether a relevant feature in one of the rows occurs just one or multiple times and it ignores the order in which the features occur. In [311], they propose the k -max pooling which returns the top k activations in the original order in the input sequence. Dynamic k -max pooling is a generalization of the k -max pooling operator where the k value is depended on the input feature map size. The CNN architectures mentioned above are rather shallow compared with the deep CNNs which are very successful in computer vision. Recently, Conneau et al. [315] implement a deep convolutional architecture which is up to 29 convolutional layers. They find that shortcut connections give better results when the network is very deep (49 layers). However, they do not achieve state-of-the-art under this setting.

6. Conclusions and outlook

Deep CNNs have made breakthroughs in processing image, video, speech and text. In this paper, we have given an extensive survey on recent advances of CNNs. We have discussed the improvements of CNN on different aspects, namely, layer design, activation function, loss function, regularization, optimization and fast computation. Beyond surveying the advances of each aspect of CNN, we have also introduced the application of CNN on many tasks, including image classification, object detection, object tracking, pose estimation, text detection, visual saliency detection, action recognition, scene labeling, speech and natural language processing.

Although CNNs have achieved great success in experimental evaluations, there are still lots of issues that deserve further investigation. Firstly, since the recent CNNs are becoming deeper and deeper, they require large-scale dataset and massive computing power for training. Manually collecting labeled dataset requires huge amounts of human efforts. Thus, it is desired to explore unsupervised learning of CNNs. Meanwhile, to speed up training procedure, although there are already some asynchronous SGD algorithms which have shown promising result by using CPU and GPU

clusters, it is still worth to develop effective and scalable parallel training algorithms. At testing time, these deep models are highly memory demanding and time-consuming, which makes them not suitable to be deployed on mobile platforms that have limited resources. It is important to investigate how to reduce the complexity and obtain fast-to-execute models without loss of accuracy.

Furthermore, one major barrier for applying CNN on a new task is that it requires considerable skill and experience to select suitable hyperparameters such as the learning rate, kernel sizes of convolutional filters, the number of layers etc.. These hyper-parameters have internal dependencies which make them particularly expensive for tuning. Recent works have shown that there exists a big room to improve current optimization techniques for learning deep CNN architectures [12,42,316].

Finally, the solid theory of CNNs is still lacking. Current CNN model works very well for various applications. However, we do not even know why and how it works essentially. It is desirable to make more efforts on investigating the fundamental principles of CNNs. Meanwhile, it is also worth exploring how to leverage natural visual perception mechanism to further improve the design of CNN. We hope that this paper not only provides a better understanding of CNNs but also facilitates future research activities and application developments in the field of CNNs.

Acknowledgment

This research was carried out at the Rapid-Rich Object Search (ROSE) Lab at the Nanyang Technological University, Singapore. The ROSE Lab is supported by the Infocomm Media Development Authority, Singapore.

References

- [1] D.H. Hubel, T.N. Wiesel, Receptive fields and functional architecture of monkey striate cortex, *J. Physiol.* (1968) 215–243.
- [2] K. Fukushima, S. Miyake, Neocognitron: a self-organizing neural network model for a mechanism of visual pattern recognition, in: *Competition and Cooperation in Neural Nets*, 1982, pp. 267–285.
- [3] B.B. Le Cun, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, Handwritten digit recognition with a back-propagation network, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 1989, pp. 396–404.
- [4] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [5] R. Hecht-Nielsen, Theory of the backpropagation neural network, *Neural Networks 1 (Supplement-1)* (1988) 445–448.
- [6] W. Zhang, K. Itoh, J. Tanida, Y. Ichioka, Parallel distributed processing model with local space-invariant interconnections and its optical architecture, *Appl. Opt.* 29 (32) (1990) 4790–4797.
- [7] X.-X. Niu, C.Y. Suen, A novel hybrid CNN-SVM classifier for recognizing handwritten digits, *Pattern Recognit.* 45 (4) (2012) 1318–1325.
- [8] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al., Imagenet large scale visual recognition challenge, *Int. J. Computer Vision (IJCV)* 115 (3) (2015) 211–252.
- [9] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [10] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [11] M.D. Zeiler, R. Fergus, Visualizing and understanding convolutional networks, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 818–833.
- [12] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [13] Y.A. LeCun, L. Bottou, G.B. Orr, K.-R. Müller, Efficient backprop, in: *Neural Networks: Tricks of the Trade - Second Edition*, 2012, pp. 9–48.
- [14] V. Nair, G.E. Hinton, Rectified linear units improve restricted Boltzmann machines, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.
- [15] T. Wang, D.J. Wu, A. Coates, A.Y. Ng, End-to-end text recognition with convolutional neural networks, in: *Proceedings of the International Conference on Pattern Recognition (ICPR)*, 2012, pp. 3304–3308.
- [16] Y. Boureau, J. Ponce, Y. LeCun, A theoretical analysis of feature pooling in visual recognition, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2010, pp. 111–118.
- [17] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, R.R. Salakhutdinov, Improving neural networks by preventing co-adaptation of feature detectors, *CoRR abs/1207.0580* (2012).
- [18] M. Lin, Q. Chen, S. Yan, Network in network, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [19] Y. Tang, Deep learning using linear support vector machines, in: *Proceedings of the International Conference on Machine Learning (ICML) Workshops*, 2013.
- [20] G. Madjarov, D. Kocev, D. Gjorgjevikj, S. Džeroski, An extensive experimental comparison of methods for multi-label learning, *Pattern Recognit.* 45 (9) (2012) 3084–3104.
- [21] R.G.J. Wijnhoven, P.H.N. de With, Fast training of object detection using stochastic gradient descent, in: *International Conference on Pattern Recognition (ICPR)*, 2010, pp. 424–427.
- [22] M. Zinkevich, M. Weimer, L. Li, A.J. Smola, Parallelized stochastic gradient descent, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2010, pp. 2595–2603.
- [23] J. Ngiam, Z. Chen, D. Chia, P.W. Koh, Q.V. Le, A.Y. Ng, Tiled convolutional neural networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2010, pp. 1279–1287.
- [24] Z. Wang, T. Oates, Encoding time series as images for visual inspection and classification using tiled convolutional neural networks, in: *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI) Workshops*, 2015.
- [25] Y. Zheng, Q. Liu, E. Chen, Y. Ge, J.L. Zhao, Time series classification using multi-channels deep convolutional neural networks, in: *Proceedings of the International Conference on Web-Age Information Management (WAIM)*, 2014, pp. 298–310.
- [26] M.D. Zeiler, D. Krishnan, G.W. Taylor, R. Fergus, Deconvolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010, pp. 2528–2535.
- [27] M.D. Zeiler, G.W. Taylor, R. Fergus, Adaptive deconvolutional networks for mid and high level feature learning, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2011, pp. 2018–2025.
- [28] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 39 (4) (2017) 640–651.
- [29] F. Visin, K. Kastner, A. Courville, Y. Bengio, M. Matteucci, K. Cho, Reseg: a recurrent neural network for object segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2015.
- [30] H. Noh, S. Hong, B. Han, Learning deconvolution network for semantic segmentation, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1520–1528.
- [31] C. Cao, X. Liu, Y. Yang, Y. Yu, J. Wang, Z. Wang, Y. Huang, L. Wang, C. Huang, W. Xu, et al., Look and think twice: capturing top-down visual attention with feedback convolutional neural networks, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 2956–2964.
- [32] J. Zhang, Z. Lin, J. Brandt, X. Shen, S. Sclaroff, Top-down neural attention by excitation backprop, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016, pp. 543–559.
- [33] Y. Zhang, E.K. Lee, E.H. Lee, U. EDU, Augmenting supervised neural networks with unsupervised objectives for large-scale image classification, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2016, pp. 612–621.
- [34] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, A. Torralba, Learning deep features for discriminative localization, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2921–2929.
- [35] A. Das, H. Agrawal, C.L. Zitnick, D. Parikh, D. Batra, Human attention in visual question answering: Do humans and deep networks look at the same regions? in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016, pp. 932–937.
- [36] C. Dong, C.C. Loy, K. He, X. Tang, Image super-resolution using deep convolutional networks, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 38 (2) (2016) 295–307.
- [37] F. Yu, V. Koltun, Multi-scale context aggregation by dilated convolutions, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [38] N. Kalchbrenner, L. Espeholt, K. Simonyan, A.v.d. Oord, A. Graves, K. Kavukcuoglu, Neural machine translation in linear time, *CoRR abs/1610.10099* (2016).
- [39] A.v.d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, K. Kavukcuoglu, Wavenet: a generative model for raw audio, *CoRR abs/1609.03499* (2016).
- [40] T. Sercu, V. Goel, Dense prediction on sequences with time-dilated convolutions for speech recognition, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS) Workshops*, 2016.
- [41] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.
- [42] C. Szegedy, S. Ioffe, V. Vanhoucke, Inception-v4, inception-resnet and the impact of residual connections on learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017, pp. 4278–4284.

- [43] A. Hyvärinen, U. Köster, Complex cell pooling and the statistics of natural images, *Network* 18 (2) (2007) 81–100.
- [44] J.B. Estrach, A. Szlam, Y. Lecun, Signal recovery from pooling representations, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2014, pp. 307–315.
- [45] L. Wan, M. Zeiler, S. Zhang, Y.L. Cun, R. Fergus, Regularization of neural networks using dropconnect, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2013, pp. 1058–1066.
- [46] D. Yu, H. Wang, P. Chen, Z. Wei, Mixed pooling for convolutional neural networks, in: *Proceedings of the Rough Sets and Knowledge Technology (RSKT)*, 2014, pp. 364–375.
- [47] M.D. Zeiler, R. Fergus, Stochastic pooling for regularization of deep convolutional neural networks, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2013.
- [48] O. Rippel, J. Snoek, R.P. Adams, Spectral representations for convolutional neural networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2015, pp. 2449–2457.
- [49] M. Mathieu, M. Henaff, Y. LeCun, Fast training of convolutional networks through FFTs, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [50] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 37 (9) (2015) 1904–1916.
- [51] S. Singh, A. Gupta, A. Efros, Unsupervised discovery of mid-level discriminative patches, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2012, pp. 73–86.
- [52] Y. Gong, L. Wang, R. Guo, S. Lazebnik, Multi-scale orderless pooling of deep convolutional activation features, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 392–407.
- [53] H. Jégou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, C. Schmid, Aggregating local image descriptors into compact codes, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 34 (9) (2012) 1704–1716.
- [54] A.L. Maas, A.Y. Hannun, A.Y. Ng, Rectifier nonlinearities improve neural network acoustic models, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2013, pp. 30–37.
- [55] M.D. Zeiler, M. Ranzato, R. Monga, M. Mao, K. Yang, Q.V. Le, P. Nguyen, A. Senior, V. Vanhoucke, J. Dean, et al., On rectified linear units for speech processing, in: *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2013, pp. 3517–3521.
- [56] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.
- [57] B. Xu, N. Wang, T. Chen, M. Li, Empirical evaluation of rectified activations in convolutional network, in: *Proceedings of the International Conference on Machine Learning (ICML) Workshop*, 2015.
- [58] D.-A. Clevert, T. Unterthiner, S. Hochreiter, Fast and accurate deep network learning by exponential linear units (elus), in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [59] I.J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio, Maxout networks, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2013, pp. 1319–1327.
- [60] J.T. Springenberg, M. Riedmiller, Improving deep neural networks with probabilistic maxout units, *CoRR abs/1312.6116* (2013).
- [61] T. Zhang, Solving large scale linear prediction problems using stochastic gradient descent algorithms, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2004.
- [62] L. Deng, The mnist database of handwritten digit images for machine learning research, *IEEE Signal Process. Mag.* 29 (6) (2012) 141–142.
- [63] W. Liu, Y. Wen, Z. Yu, M. Yang, Large-margin softmax loss for convolutional neural networks, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2016, pp. 507–516.
- [64] J. Bromley, J.W. Bentz, L. Bottou, I. Guyon, Y. LeCun, C. Moore, E. Säckinger, R. Shah, Signature verification using a siamese time delay neural network, *Int. J. Pattern Recognit. Artif. Intell. (IJPRAI)* 7 (4) (1993) 669–688.
- [65] S. Chopra, R. Hadsell, Y. LeCun, Learning a similarity metric discriminatively, with application to face verification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 539–546.
- [66] R. Hadsell, S. Chopra, Y. LeCun, Dimensionality reduction by learning an invariant mapping, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2006, pp. 1735–1742.
- [67] U. Shaham, R.R. Lederman, Learning by coincidence: siamese networks and common variable learning, *Pattern Recognit.* (2017).
- [68] J. Lin, O. Morere, V. Chandrasekhar, A. Veillard, H. Goh, Deephash: getting regularization, depth and fine-tuning right, in: *Proceedings of the International Conference on Multimedia Retrieval (ICMR)*, 2017, pp. 133–141.
- [69] F. Schroff, D. Kalenichenko, J. Philbin, Facenet: a unified embedding for face recognition and clustering, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [70] H. Liu, Y. Tian, Y. Yang, L. Pang, T. Huang, Deep relative distance learning: tell the difference between similar vehicles, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2167–2175.
- [71] S. Ding, L. Lin, G. Wang, H. Chao, Deep feature learning with relative distance comparison for person re-identification, *Pattern Recognit.* 48 (10) (2015) 2993–3003.
- [72] Z. Liu, P. Luo, S. Qiu, X. Wang, X. Tang, Deepfashion: powering robust clothes recognition and retrieval with rich annotations, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.
- [73] D.P. Kingma, M. Welling, Auto-encoding variational bayes, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [74] D.J. Im, S. Ahn, R. Memisevic, Y. Bengio, Denoising criterion for variational auto-encoding framework, in: *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, 2017, pp. 2059–2065.
- [75] D.P. Kingma, S. Mohamed, D.J. Rezende, M. Welling, Semi-supervised learning with deep generative models, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 3581–3589.
- [76] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 2672–2680.
- [77] M. Mirza, S. Osindero, Conditional generative adversarial nets, *CoRR abs/1411.1784* (2014).
- [78] P. Vincent, H. Larochelle, Y. Bengio, P.-A. Manzagol, Extracting and composing robust features with denoising autoencoders, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2008, pp. 1096–1103.
- [79] W.W. Ng, G. Zeng, J. Zhang, D.S. Yeung, W. Pedrycz, Dual autoencoders features for imbalance classification problem, *Pattern Recognit.* 60 (2016) 875–889.
- [80] J. Mehta, A. Majumdar, Rodeo: robust de-aliasing autoencoder for real-time medical image reconstruction, *Pattern Recognit.* 63 (2017) 499–510.
- [81] B.A. Olshausen, et al., Emergence of simple-cell receptive field properties by learning a sparse code for natural images, *Nature* 381 (6583) (1996) 607.
- [82] H. Lee, A. Battle, R. Raina, A.Y. Ng, Efficient sparse coding algorithms, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2006, pp. 801–808.
- [83] S. Eslami, N. Heess, T. Weber, Y. Tassa, K. Kavukcuoglu, G.E. Hinton, Attend, infer, repeat: fast scene understanding with generative models, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 3225–3233.
- [84] K. Sohn, H. Lee, X. Yan, Learning structured output representation using deep conditional generative models, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2015, pp. 3483–3491.
- [85] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, H. Lee, Generative adversarial text to image synthesis, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2016, pp. 1060–1069.
- [86] E.L. Denton, S. Chintala, R. Fergus, et al., Deep generative image models using a Laplacian pyramid of adversarial networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2015, pp. 1486–1494.
- [87] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, X. Chen, Improved techniques for training GANs, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 2226–2234.
- [88] A. Dosovitskiy, T. Brox, Generating images with perceptual similarity metrics based on deep networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 658–666.
- [89] A.N. Tikhonov, On the stability of inverse problems, in: *Dokl. Akad. Nauk SSSR*, 39, 1943, pp. 195–198.
- [90] S. Wang, C. Manning, Fast dropout training, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2013, pp. 118–126.
- [91] J. Ba, B. Frey, Adaptive dropout for training deep neural networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2013, pp. 3084–3092.
- [92] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, C. Bregler, Efficient object localization using convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 648–656.
- [93] H. Yang, I. Patras, Mirror, mirror on the wall, tell me, is the error small? in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 4685–4693.
- [94] S. Xie, Z. Tu, Holistically-nested edge detection, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1395–1403.
- [95] J. Salamon, J.P. Bello, Deep convolutional neural networks and data augmentation for environmental sound classification, *Signal Process. Lett. (SPL)* 24 (3) (2017) 279–283.
- [96] D. Eigen, R. Fergus, Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 2650–2658.
- [97] M. Paulin, J. Revaud, Z. Harchaoui, F. Perronnin, C. Schmid, Transformation pursuit for image classification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 3646–3653.
- [98] S. Hauberg, O. Freifeld, A.B.L. Larsen, J.W. Fisher III, L.K. Hansen, Dreaming more data: Class-dependent distributions over diffeomorphisms for learned data augmentation, in: *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016, pp. 342–350.
- [99] S. Xie, T. Yang, X. Wang, Y. Lin, Hyper-class augmented and regularized deep learning for fine-grained image classification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 2645–2654.
- [100] Z. Xu, S. Huang, Y. Zhang, D. Tao, Augmenting strong supervision using web data for fine-grained categorization, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 2524–2532.

- [101] A. Choromanska, M. Henaff, M. Mathieu, G.B. Arous, Y. LeCun, The loss surfaces of multilayer networks, in: Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS), 2015.
- [102] D. Mishkin, J. Matas, All you need is a good init, in: Proceedings of the International Conference on Learning Representations (ICLR), 2016.
- [103] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: Proceedings of the International Conference on Machine Learning (ICML), 2013, pp. 1139–1147.
- [104] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS), 2010, pp. 249–256.
- [105] A.M. Saxe, J.L. McClelland, S. Ganguli, Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014.
- [106] C. Doersch, A. Gupta, A.A. Efros, Unsupervised visual representation learning by context prediction, in: Proceedings of the International Conference on Computer Vision (ICCV), 2015, pp. 1422–1430.
- [107] P. Agrawal, J. Carreira, J. Malik, Learning to see by moving, in: Proceedings of the International Conference on Computer Vision (ICCV), 2015, pp. 37–45.
- [108] N. Qian, On the momentum term in gradient descent learning algorithms, *Neural Netw.* 12 (1) (1999) 145–151.
- [109] D. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Proceedings of the International Conference on Learning Representations (ICLR), 2015.
- [110] I. Loshchilov, F. Hutter, Sgdr: Stochastic gradient descent with warm restarts, in: Proceedings of the International Conference on Learning Representations (ICLR), 2017.
- [111] T. Schaul, S. Zhang, Y. LeCun, No more pesky learning rates, in: Proceedings of the International Conference on Machine Learning (ICML), 2013, pp. 343–351.
- [112] S. Zhang, A.E. Choromanska, Y. LeCun, Deep learning with elastic averaging SGD, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2015, pp. 685–693.
- [113] B. Recht, C. Re, S. Wright, F. Niu, Hogwild: a lock-free approach to parallelizing stochastic gradient descent, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2011, pp. 693–701.
- [114] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, A. Senior, P. Tucker, K. Yang, Q.V. Le, et al., Large scale distributed deep networks, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2012, pp. 1232–1240.
- [115] T. Paine, H. Jin, J. Yang, Z. Lin, T. Huang, GPU asynchronous stochastic gradient descent to speed up neural network training, *CoRR* (2011). [abs/1107.2490](https://arxiv.org/abs/1107.2490)
- [116] Y. Zhuang, W.-S. Chin, Y.-C. Juan, C.-J. Lin, A fast parallel SGD for matrix factorization in shared memory systems, in: Proceedings of the ACM conference on Recommender systems RecSys, 2013, pp. 249–256.
- [117] Y. Yao, L. Rosasco, A. Caponnetto, On early stopping in gradient descent learning, *Constructive Approx.* 26 (2) (2007) 289–315.
- [118] L. Prechelt, Early stopping - but when? in: *Neural Networks: Tricks of the Trade - Second Edition*, 2012, pp. 53–67.
- [119] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning requires rethinking generalization, in: Proceedings of the International Conference on Learning Representations (ICLR), 2017.
- [120] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, *J. Mach. Learn. Res. (JMLR)* (2015) 448–456.
- [121] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [122] R.K. Srivastava, K. Greff, J. Schmidhuber, Training very deep networks, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2015, pp. 2377–2385.
- [123] K. He, X. Zhang, S. Ren, J. Sun, Identity mappings in deep residual networks, in: Proceedings of the European Conference on Computer Vision (ECCV), 2016, pp. 630–645.
- [124] F. Shen, R. Gan, G. Zeng, Weighted residuals for very deep networks, in: Proceedings of the International Conference on Systems and Informatics (ICSAI), 2016, pp. 936–941.
- [125] S. Zagoruyko, N. Komodakis, Wide residual networks, in: Proceedings of the British Machine Vision Conference (BMVC), 2016, pp. 871–8712.
- [126] S. Singh, D. Hoiem, D. Forsyth, Swapout: learning an ensemble of deep architectures, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2016, pp. 28–36.
- [127] S. Targ, D. Almeida, K. Lyman, Resnet in resnet: generalizing residual architectures, *CoRR* (2016). [abs/1603.08029](https://arxiv.org/abs/1603.08029)
- [128] K. Zhang, M. Sun, T.X. Han, X. Yuan, L. Guo, T. Liu, Residual networks of residual networks: multilevel residual networks, *IEEE Trans. Circuits Syst. Video Technol. (TCSVT)* PP (99) (2016) 1.
- [129] G. Huang, Z. Liu, K.Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 4700–4708.
- [130] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, E. Shelhamer, Cudnn: efficient primitives for deep learning [abs/1410.0759](https://arxiv.org/abs/1410.0759) (2014).
- [131] N. Vasilache, J. Johnson, M. Mathieu, S. Chintala, S. Piantino, Y. LeCun, Fast convolutional nets with fbfft: aGPU performance evaluation, in: Proceedings of the International Conference on Learning Representations (ICLR), 2015.
- [132] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, Y. LeCun, Overfeat: integrated recognition, localization and detection using convolutional networks (2014).
- [133] A. Lavin, Fast algorithms for convolutional neural networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 4013–4021.
- [134] T.N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, B. Ramabhadran, Low-rank matrix factorization for deep neural network training with high-dimensional output targets, in: Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2013, pp. 6655–6659.
- [135] J. Xue, J. Li, Y. Gong, Restructuring of deep neural network acoustic models with singular value decomposition, in: Proceedings of the International Speech Communication Association (INTERSPEECH), 2013, pp. 2365–2369.
- [136] M. Denil, B. Shakibi, L. Dinh, N. de Freitas, et al., Predicting parameters in deep learning, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2013, pp. 2148–2156.
- [137] E.L. Denton, W. Zaremba, J. Bruna, Y. LeCun, R. Fergus, Exploiting linear structure within convolutional networks for efficient evaluation, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2014, pp. 1269–1277.
- [138] M. Jaderberg, A. Vedaldi, A. Zisserman, Speeding up convolutional neural networks with low rank expansions, in: Proceedings of the British Machine Vision Conference (BMVC), 2014.
- [139] A. Novikov, D. Podoprikin, A. Osokin, D.P. Vetrov, Tensorizing neural networks, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2015, pp. 442–450.
- [140] I.V. Oseledets, Tensor-train decomposition, *SIAM J. Sci. Comput.* 33 (5) (2011) 2295–2317.
- [141] Q. Le, T. Sarlós, A. Smola, Fastfood: approximating kernel expansions in loglinear time, in: Proceedings of the International Conference on Machine Learning (ICML), 2013, pp. 85, 2013.
- [142] A. Dasgupta, R. Kumar, T. Sarlós, Fast locality-sensitive hashing, in: Proceedings of the International Conference on Knowledge Discovery and Data Mining (SIGKDD), 2011, pp. 1073–1081.
- [143] F.X. Yu, S. Kumar, Y. Gong, S.-F. Chang, Circulant binary embedding, in: Proceedings of the International Conference on Machine Learning (ICML), 2014, pp. 946–954.
- [144] Y. Cheng, F.X. Yu, R.S. Feris, S. Kumar, A. Choudhary, S.-F. Chang, An exploration of parameter redundancy in deep networks with circulant projections, in: Proceedings of the International Conference on Computer Vision (ICCV), 2015, pp. 2857–2865.
- [145] M. Moczulski, M. Denil, J. Appleby, N. de Freitas, Acdc: a structured efficient linear layer, in: Proceedings of the International Conference on Learning Representations (ICLR), 2016.
- [146] S. Han, H. Mao, W.J. Dally, Deep compression: compressing deep neural network with pruning, trained quantization and Huffman coding, in: Proceedings of the International Conference on Learning Representations (ICLR), 2016.
- [147] M. Kim, P. Smaragdakis, Bitwise neural networks, in: Proceedings of the International Conference on Machine Learning (ICML) Workshops, 2016.
- [148] M. Rastegari, V. Ordonez, J. Redmon, A. Farhadi, Xnor-net: imagenet classification using binary convolutional neural networks, in: Proceedings of the European Conference on Computer Vision (ECCV), 2016, pp. 525–542.
- [149] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, Y. Zou, Dorefa-net: training low bandwidth convolutional neural networks with low bandwidth gradients, *CoRR* (2016). [abs/1606.06160](https://arxiv.org/abs/1606.06160)
- [150] M. Courbariaux, Y. Bengio, Binarynet: training deep neural networks with weights and activations constrained to +1 or -1, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2016.
- [151] G.J. Sullivan, Efficient scalar quantization of exponential and Laplacian random variables, *IEEE Trans. Inf. Theory* 42 (5) (1996) 1365–1374.
- [152] Y. Gong, L. Liu, M. Yang, L. Bourdev, Compressing deep convolutional networks using vector quantization, in: *arXiv preprint arXiv:1412.6115*, volume [abs/1412.6115](https://arxiv.org/abs/1412.6115), 2014.
- [153] Y. Chen, T. Guan, C. Wang, Approximate nearest neighbor search by residual vector quantization, *Sensors* 10 (12) (2010) 11259–11273.
- [154] W. Zhou, Y. Lu, H. Li, Q. Tian, Scalar quantization for large scale image search, in: Proceedings of the 20th ACM International Conference on Multimedia, 2012, pp. 169–178.
- [155] L.Y. Pratt, Comparing biases for minimal network construction with back-propagation, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 1988, pp. 177–185.
- [156] S. Han, J. Pool, J. Tran, W. Dally, Learning both weights and connections for efficient neural network, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2015, pp. 1135–1143.
- [157] Y. Guo, A. Yao, Y. Chen, Dynamic network surgery for efficient DNNs, in: Proceedings of the Advances in Neural Information Processing Systems (NIPS), 2016, pp. 1379–1387.
- [158] T.-J. Yang, Y.-H. Chen, V. Sze, Designing energy-efficient convolutional neural networks using energy-aware pruning, *CoRR* [abs/1611.05128](https://arxiv.org/abs/1611.05128) (2016).
- [159] H. Hu, R. Peng, Y.-W. Tai, C.-K. Tang, Network trimming: a data-driven neuron pruning approach towards efficient deep architectures, volume [abs/1607.03250](https://arxiv.org/abs/1607.03250), 2016.
- [160] S. Srinivas, R.V. Babu, Data-free parameter pruning for deep neural networks, in: Proceedings of the British Machine Vision Conference (BMVC), 2015.
- [161] Z. Mariet, S. Sra, Diversity networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2015.
- [162] W. Chen, J.T. Wilson, S. Tyree, K.Q. Weinberger, Y. Chen, Compressing neural networks with the hashing trick, in: Proceedings of the International Conference on Machine Learning (ICML), 2015, pp. 2285–2294.

- [163] Q. Shi, J. Petterson, G. Dror, J. Langford, A. Smola, S. Vishwanathan, Hash kernels for structured data, *J. Mach. Learn. Res. (JMLR)* 10 (2009) 2615–2637.
- [164] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, J. Attenberg, Feature hashing for large scale multitask learning, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2009, pp. 1113–1120.
- [165] B. Liu, M. Wang, H. Foroosh, M. Tappen, M. Pensky, Sparse convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 806–814.
- [166] W. Wen, C. Wu, Y. Wang, Y. Chen, H. Li, Learning structured sparsity in deep neural networks, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 2074–2082.
- [167] H. Bagherinezhad, M. Rastegari, A. Farhadi, Lcnn: Lookup-based convolutional neural network, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [168] M. Egmont-Petersen, D. de Ridder, H. Handels, Image processing with neural networks a review, *Pattern Recognit.* 35 (10) (2002) 2279–2301.
- [169] K. Nogueira, O.A. Penatti, J.A. dos Santos, Towards better exploiting convolutional neural networks for remote sensing scene classification, *Pattern Recognit.* 61 (2017) 539–556.
- [170] Z. Zuo, G. Wang, B. Shuai, L. Zhao, Q. Yang, Exemplar based deep discriminative and shareable feature learning for scene image classification, *Pattern Recognit.* 48 (10) (2015) 3004–3015.
- [171] A.T. Lopes, E. de Aguiar, A.F. De Souza, T. Oliveira-Santos, Facial expression recognition with convolutional neural networks: coping with few data and the training sample order, *Pattern Recognit.* 61 (2017) 610–628.
- [172] M. Everingham, S.A. Eslami, L. Van Gool, C.K. Williams, J. Winn, A. Zisserman, The pascal visual object classes challenge: a retrospective, *Int. J. Conflict Violence (IJCV)* 111 (1) (2015) 98–136.
- [173] A.-M. Tsch, S. Herbin, J.-Y. Audibert, Semantic hierarchies for image annotation: a survey, *Pattern Recognit.* 45 (1) (2012) 333–345.
- [174] N. Srivastava, R.R. Salakhutdinov, Discriminative transfer learning with tree-based priors, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2013, pp. 2094–2102.
- [175] Z. Wang, X. Wang, G. Wang, Learning fine-grained features via a CNN tree for large-scale classification, *CoRR abs/1511.04534* (2015).
- [176] T. Xiao, J. Zhang, K. Yang, Y. Peng, Z. Zhang, Error-driven incremental learning in deep convolutional neural network for large-scale image classification, in: *Proceedings of the ACM Multimedia Conference*, 2014, pp. 177–186.
- [177] Z. Yan, V. Jagadeesh, D. DeCoste, W. Di, R. Piramuthu, Hd-cnn: hierarchical deep convolutional neural network for image classification, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, pp. 2740–2748.
- [178] T. Berg, J. Liu, S.W. Lee, M.L. Alexander, D.W. Jacobs, P.N. Belhumeur, Birdsnap: large-scale fine-grained visual categorization of birds, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 2019–2026.
- [179] A. Khosla, N. Jayadevaprakash, B. Yao, F.-F. Li, Novel dataset for fine-grained image categorization: stanford dogs, in: *Proceedings of the IEEE International Conference on Computer Vision (CVPR Workshops)*, 2, 2011, p. 1.
- [180] L. Yang, P. Luo, C.C. Loy, X. Tang, A large-scale car dataset for fine-grained categorization and verification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3973–3981.
- [181] M. Minervini, A. Fischbach, H. Scharr, S.A. Tsafaris, Finely-grained annotated datasets for image-based plant phenotyping, *Pattern Recognit. Lett.* 81 (2016) 80–89.
- [182] G.-S. Xie, X.-Y. Zhang, W. Yang, M.-L. Xu, S. Yan, C.-L. Liu, Lg-cnn: from local parts to global discrimination for fine-grained recognition, *Pattern Recognit.* 71 (2017) 118–131.
- [183] S. Branson, G. Van Horn, P. Perona, S. Belongie, Improved bird species recognition using pose normalized deep convolutional nets, in: *Proceedings of the British Machine Vision Conference (BMVC)*, 2014.
- [184] N. Zhang, J. Donahue, R. Girshick, T. Darrell, Part-based r-cnns for fine-grained category detection, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 834–849.
- [185] J.R. Uijlings, K.E. van de Sande, T. Gevers, A.W. Smeulders, Selective search for object recognition, *Int. J. Conflict Violence (IJCV)* 104 (2) (2013) 154–171.
- [186] D. Lin, X. Shen, C. Lu, J. Jia, Deep lac: deep localization, alignment and classification for fine-grained recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1666–1674.
- [187] J.P. Pluim, J.A. Maintz, M. Viergever, et al., Mutual-information-based registration of medical images: a survey, *IEEE Trans. Med. Imaging* 22 (8) (2003) 986–1004.
- [188] J. Krause, T. Gebru, J. Deng, L.-J. Li, L. Fei-Fei, Learning features and parts for fine-grained recognition, in: *Proceedings of the International Conference on Pattern Recognition (ICPR)*, 2014, pp. 26–33.
- [189] J. Krause, H. Jin, J. Yang, L. Fei-Fei, Fine-grained recognition without part annotations, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 5546–5555.
- [190] Y. Zhang, X.-S. Wei, J. Wu, J. Cai, J. Lu, V.-A. Nguyen, M.N. Do, Weakly supervised fine-grained categorization with part-based image representation, *IEEE Trans. Image Process.* 25 (4) (2016) 1713–1725.
- [191] T. Xiao, Y. Xu, K. Yang, J. Zhang, Y. Peng, Z. Zhang, The application of two-level attention models in deep convolutional neural network for fine-grained image classification, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 842–850.
- [192] T.-Y. Lin, A. RoyChowdhury, S. Maji, Bilinear CNN models for fine-grained visual recognition, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1449–1457.
- [193] D.T. Nguyen, W. Li, P.O. Ogunbona, Human detection from images and videos: a survey, *Pattern Recognit.* 51 (2016) 148–175.
- [194] Y. Li, S. Wang, Q. Tian, X. Ding, Feature representation for statistical-learning-based object detection: a review, *Pattern Recognit.* 48 (11) (2015) 3542–3559.
- [195] M. Pedersoli, A. Vedaldi, J. Gonzalez, X. Roca, A coarse-to-fine approach for fast deformable object detection, *Pattern Recognit.* 48 (5) (2015) 1844–1853.
- [196] S.J. Nowlan, J.C. Platt, A convolutional neural network hand tracker, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 1994, pp. 901–908.
- [197] R. Girshick, F. Iandola, T. Darrell, J. Malik, Deformable part models are convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 437–446.
- [198] R. Vaillant, C. Monrocq, Y. Le Cun, Original approach for the localisation of objects in images, *IEE Proc.-Vis. Image Signal Process.* 141 (4) (1994) 245–250.
- [199] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft Coco: Common Objects in Context, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [200] I. Endres, D. Hoiem, Category independent object proposals, *IEEE Trans. Pattern Anal. Mach.Intell. (PAMI)* 36 (2) (2014) 222–234.
- [201] L. Gómez, D. Karatzas, Textproposals: a text-specific selective search algorithm for word spotting in the wild, *Pattern Recognit.* 70 (2017) 60–74.
- [202] R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587.
- [203] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, *IEEE Trans. Pattern Anal. Mach.Intell. (PAMI)* 37 (9) (2015) 1904–1916.
- [204] S. Ren, K. He, R. Girshick, J. Sun, Faster r-CNN: towards real-time object detection with region proposal networks, *IEEE Trans. Pattern Anal. Mach.Intell. (PAMI)* 39 (6) (2017) 1137–1149.
- [205] S. Gidaris, N. Komodakis, Object detection via a multi-region and semantic segmentation-aware cnn model, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1134–1142.
- [206] D. Yoo, S. Park, J.-Y. Lee, A.S. Paek, I. So Kweon, Attentionnet: Aggregating weak directions for accurate object detection, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 2659–2667.
- [207] P.F. Felzenszwalb, R.B. Girshick, D. McAllester, D. Ramanan, Object detection with discriminatively trained part-based models, *IEEE Trans. Pattern Anal. Mach.Intell. (PAMI)* 32 (9) (2010) 1627–1645.
- [208] E. Simo-Serra, E. Trulls, L. Ferraz, I. Kokkinos, F. Moreno-Noguer, Fracking deep convolutional image descriptors, *CoRR abs/1412.6537* (2014).
- [209] A. Shrivastava, A. Gupta, R. Girshick, Training region-based object detectors with online hard example mining, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 761–769.
- [210] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: unified, real-time object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [211] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, SSD: single shot multi-box detector, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37.
- [212] Y. Lu, T. Javidi, S. Lazebnik, Adaptive object detection using adjacency and zoom prediction, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2351–2359.
- [213] K. Zhang, H. Song, Real-time visual tracking via online weighted multiple instance learning, *Pattern Recognit.* 46 (1) (2013) 397–411.
- [214] S. Zhang, H. Yao, X. Sun, X. Lu, Sparse coding based visual tracking: review and experimental comparison, *Pattern Recognit.* 46 (7) (2013) 1772–1788.
- [215] S. Zhang, J. Wang, Z. Wang, Y. Gong, Y. Liu, Multi-target tracking by learning local-to-global trajectory models, *Pattern Recognit.* 48 (2) (2015) 580–590.
- [216] J. Fan, W. Xu, Y. Wu, Y. Gong, Human tracking using convolutional neural networks, *IEEE Trans. Neural Netw. (TNN)* 21 (10) (2010) 1610–1623.
- [217] H. Li, Y. Li, F. Porikli, Deeptack: learning discriminative feature representations by convolutional neural networks for visual tracking, in: *Proceedings of the British Machine Vision Conference (BMVC)*, 2014.
- [218] Y. Chen, X. Yang, B. Zhong, S. Pan, D. Chen, H. Zhang, Cnntracker: online discriminative object tracking via deep convolutional neural network, *Appl. Soft Comput.* 38 (2016) 1088–1098.
- [219] S. Hong, T. You, S. Kwak, B. Han, Online tracking by learning discriminative saliency map with convolutional neural network, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2015, pp. 597–606.
- [220] M. Patacchiola, A. Cangelosi, Head pose estimation in the wild using convolutional neural networks and adaptive gradient methods, *Pattern Recognit.* 71 (2017) 132–143.
- [221] K. Nishi, J. Miura, Generation of human depth images with body part labels for complex human pose recognition, *Pattern Recognit.* (2017).
- [222] A. Toshev, C. Szegedy, Deeppose: human pose estimation via deep neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1653–1660.
- [223] A. Jain, J. Tompson, M. Andriluka, G.W. Taylor, C. Bregler, Learning human pose estimation features with convolutional networks, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.

- [224] J.J. Tompson, A. Jain, Y. LeCun, C. Bregler, Joint training of a convolutional network and a graphical model for human pose estimation, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 1799–1807.
- [225] X. Chen, A.L. Yuille, Articulated pose estimation by a graphical model with image dependent pairwise relations, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 1736–1744.
- [226] X. Chen, A. Yuille, Parsing occluded people by flexible compositions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3945–3954.
- [227] X. Fan, K. Zheng, Y. Lin, S. Wang, Combining local appearance and holistic view: dual-source deep neural networks for human pose estimation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1347–1355.
- [228] A. Jain, J. Tompson, Y. LeCun, C. Bregler, Modeep: A deep learning framework using motion features for human pose estimation, in: *Proceedings of the Asian Conference on Computer Vision (ACCV)*, 2014, pp. 302–315.
- [229] Y.Y. Tang, S.-W. Lee, C.Y. Suen, Automatic document processing: a survey, *Pattern Recognit.* 29 (12) (1996) 1931–1952.
- [230] A. Vinciarelli, A survey on off-line cursive word recognition, *Pattern Recognit.* 35 (7) (2002) 1433–1446.
- [231] K. Jung, K.I. Kim, A.K. Jain, Text information extraction in images and video: a survey, *Pattern Recognit.* 37 (5) (2004) 977–997.
- [232] S. Eskenazi, P. Gomez-Krämer, J.-M. Ogier, A comprehensive survey of mostly textual document segmentation algorithms since 2008, *Pattern Recognit.* 64 (2017) 1–14.
- [233] X. Bai, B. Shi, C. Zhang, X. Cai, L. Qi, Text/non-text image classification in the wild with convolutional neural networks, *Pattern Recognit.* 66 (2017) 437–446.
- [234] L. Gomez, A. Nicolaou, D. Karatzas, Improving patch-based scene text script identification with ensembles of conjoined networks, *Pattern Recognit.* 67 (2017) 85–96.
- [235] M. Delakis, C. Garcia, Text detection with convolutional neural networks, in: *Proceedings of the International Conference on Computer Vision Theory and Applications (VISAPP)*, 2008, pp. 290–294.
- [236] H. Xu, F. Su, Robust seed localization and growing with deep convolutional features for scene text detection, in: *Proceedings of the International Conference on Multimedia Retrieval (ICMR)*, 2015, pp. 387–394.
- [237] W. Huang, Y. Qiao, X. Tang, Robust scene text detection with convolution neural network induced msr trees, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 497–511.
- [238] C. Zhang, C. Yao, B. Shi, X. Bai, Automatic discrimination of text and non-text natural images, in: *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2015, pp. 886–890.
- [239] I.J. Goodfellow, J. Ibarz, S. Arnaud, V. Shet, Multi-digit number recognition from street view imagery using deep convolutional neural networks, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [240] M. Jaderberg, K. Simonyan, A. Vedaldi, A. Zisserman, Deep structured output learning for unconstrained text recognition, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [241] P. He, W. Huang, Y. Qiao, C.C. Loy, X. Tang, Reading scene text in deep convolutional sequences, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016, pp. 3501–3508.
- [242] F.A. Gers, J. Schmidhuber, F. Cummins, Learning to forget: continual prediction with lstm, *Neural Comput.* 12 (10) (2000) 2451–2471.
- [243] B. Shi, X. Bai, C. Yao, An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition, *CoRR abs/1507.05717* (2015).
- [244] M. Jaderberg, A. Vedaldi, A. Zisserman, Deep features for text spotting, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 512–528.
- [245] M. Jaderberg, K. Simonyan, A. Vedaldi, A. Zisserman, Reading text in the wild with convolutional neural networks, volume 116, 2016, pp. 1–20.
- [246] L. Wang, H. Lu, X. Ruan, M.-H. Yang, Deep networks for saliency detection via local estimation and global search, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3183–3192.
- [247] R. Zhao, W. Ouyang, H. Li, X. Wang, Saliency detection by multi-context deep learning, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1265–1274.
- [248] G. Li, Y. Yu, Visual saliency based on multiscale deep features, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 5455–5463.
- [249] N. Liu, J. Han, D. Zhang, S. Wen, T. Liu, Predicting eye fixations using convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 362–370.
- [250] S. He, et al., Supercnn: a superpixelwise convolutional neural network for salient object detection, *Inter. J. Comput. Vis.* 115 (3) (2015) 330–344.
- [251] E. Vig, M. Dorr, D. Cox, Large-scale optimization of hierarchical features for saliency prediction in natural images, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 2798–2805.
- [252] M. Kümmerer, L. Theis, M. Bethge, Deep gaze i: boosting saliency prediction with feature maps trained on imagenet, in: *Proceedings of the International Conference on Learning Representations (ICLR) Workshops*, 2015.
- [253] J. Pan, X. Gir-i Nieto, End-to-end convolutional network for saliency prediction, *CoRR abs/1507.01422* (2015).
- [254] G. Guo, A. Lai, A survey on still image based human action recognition, *Pattern Recognit.* 47 (10) (2014) 3343–3361.
- [255] L.L. Presti, M. La Cascia, 3D skeleton-based human action classification: a survey, *Pattern Recognit.* 53 (2016) 130–147.
- [256] J. Zhang, W. Li, P.O. Ogunbona, P. Wang, C. Tang, Rgb-d-based action recognition datasets: a survey, *Pattern Recognit.* 60 (2016) 86–105.
- [257] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, T. Darrell, Decaf: a deep convolutional activation feature for generic visual recognition, 2014.
- [258] M. Oquab, L. Bottou, I. Laptev, J. Sivic, Learning and transferring mid-level image representations using convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1717–1724.
- [259] G. Gkioxari, R. Girshick, J. Malik, Actions and attributes from wholes and parts, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 2470–2478.
- [260] L. Pishchulin, M. Andriluka, P. Gehler, B. Schiele, Poselet conditioned pictorial structures, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 588–595.
- [261] G. Gkioxari, R.B. Girshick, J. Malik, Contextual action recognition with r⁺CNN, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 1080–1088.
- [262] G. Gkioxari, R. Girshick, J. Malik, Actions and attributes from wholes and parts, in: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 2470–2478.
- [263] Y. Zhang, L. Cheng, J. Wu, J. Cai, M.N. Do, J. Lu, Action recognition in still images with minimum annotation efforts, *IEEE Trans. Image Process.* 25 (11) (2016) 5479–5490.
- [264] L. Wang, L. Ge, R. Li, Y. Fang, Three-stream CNNs for action recognition, *Pattern Recognit. Lett.* 92 (2017) 33–40.
- [265] S. Ji, W. Xu, M. Yang, K. Yu, 3D convolutional neural networks for human action recognition, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 35 (1) (2013) 221–231.
- [266] D. Tran, L. Bourdev, R. Fergus, L. Torresani, M. Paluri, Learning spatiotemporal features with 3d convolutional networks, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 4489–4497.
- [267] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, L. Fei-Fei, Large-scale video classification with convolutional neural networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1725–1732.
- [268] K. Simonyan, A. Zisserman, Two-stream convolutional networks for action recognition in videos, in: Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, K. Weinberger (Eds.), *Proceedings of the Advances in Neural Information Processing Systems (NIPS)*, 2014, pp. 568–576.
- [269] G. Chéron, I. Laptev, C. Schmid, P-CNN: pose-based CNN features for action recognition, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015, pp. 3218–3226.
- [270] J. Donahue, L. Anne Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, K. Saenko, T. Darrell, Long-term recurrent convolutional networks for visual recognition and description, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 39 (4) (2017) 677–691.
- [271] K.-S. Fu, J. Mui, A survey on image segmentation, *Pattern Recognit.* 13 (1) (1981) 3–16.
- [272] Q. Zhou, B. Zheng, W. Zhu, L.J. Latecki, Multi-scale context for scene labeling via flexible segmentation graph, *Pattern Recognit.* 59 (2016) 312–324.
- [273] F. Liu, G. Lin, C. Shen, CRF learning with CNN features for image segmentation, *Pattern Recognit.* 48 (10) (2015) 2983–2992.
- [274] S. Bu, P. Han, Z. Liu, J. Han, Scene parsing using inference embedded deep networks, *Pattern Recognit.* 59 (2016) 188–198.
- [275] B. Peng, L. Zhang, D. Zhang, A survey of graph theoretical approaches to image segmentation, *Pattern Recognit.* 46 (3) (2013) 1020–1038.
- [276] C. Farabet, C. Couprie, L. Najman, Y. LeCun, Learning hierarchical features for scene labeling, *IEEE Trans. Pattern Anal. Mach. Intell. (PAMI)* 35 (8) (2013) 1915–1929.
- [277] C. Couprie, C. Farabet, L. Najman, Y. LeCun, Indoor semantic segmentation using depth information, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2013.
- [278] P. Pinheiro, R. Collobert, Recurrent convolutional neural networks for scene labeling, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2014, pp. 82–90.
- [279] B. Shuai, G. Wang, Z. Zuo, B. Wang, L. Zhao, Integrating parametric and non-parametric models for scene labeling, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 4249–4258.
- [280] B. Shuai, Z. Zuo, W. Gang, Quaddirectional 2d-recurrent neural networks for image labeling 22(11) (2015b) 1990–1994.
- [281] B. Shuai, Z. Zuo, G. Wang, B. Wang, Dag-recurrent neural networks for scene labeling, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3620–3629.
- [282] M. Mostajabi, P. Yadollahpour, G. Shakhnarovich, Feedforward semantic segmentation with zoom-out features, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3376–3385.
- [283] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, A.L. Yuille, Semantic image segmentation with deep convolutional nets and fully connected crfs, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

- [284] M. El Ayadi, M.S. Kamel, F. Karray, Survey on speech emotion recognition: features, classification schemes, and databases, *Pattern Recognit.* 44 (3) (2011) 572–587.
- [285] L. Deng, P. Kenny, M. Lennig, V. Gupta, F. Seitz, P. Mermelstein, Phone-mic hidden Markov models with continuous mixture output densities for large vocabulary word recognition, *IEEE Trans. Signal Process.* 39 (7) (1991) 1677–1681.
- [286] G. Hinton, L. Deng, D. Yu, G.E. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T.N. Sainath, et al., Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups, *IEEE Signal Process. Mag.* 29 (6) (2012) 82–97.
- [287] L. Deng, J. Li, J.-T. Huang, K. Yao, D. Yu, F. Seide, M. Seltzer, G. Zweig, X. He, J. Williams, et al., Recent advances in deep learning for speech research at Microsoft, in: *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2013, pp. 8604–8608.
- [288] K. Yao, D. Yu, F. Seide, H. Su, L. Deng, Y. Gong, Adaptation of context-dependent deep neural networks for automatic speech recognition, in: *Proceedings of the Spoken Language Technology (SLT)*, 2012, pp. 366–369.
- [289] O. Abdel-Hamid, A.-r. Mohamed, H. Jiang, G. Penn, Applying convolutional neural networks concepts to hybrid nn-hmm model for speech recognition, in: *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2012, pp. 4277–4280.
- [290] O. Abdel-Hamid, A.-R. Mohamed, H. Jiang, L. Deng, G. Penn, D. Yu, Convolutional neural networks for speech recognition, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2014.
- [291] D. Palaz, R. Collobert, M.M. Doss, Estimating phoneme class conditional probabilities from raw speech signal using convolutional neural networks, in: *Proceedings of the International Speech Communication Association (INTERSPEECH)*, 2013, pp. 1766–1770.
- [292] Y. Hoshen, R.J. Weiss, K.W. Wilson, Speech acoustic modeling from raw multichannel waveforms, in: *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2015, pp. 4624–4628.
- [293] D. Amodei, R. Anubhai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, J. Chen, M. Chrzanowski, A. Coates, G. Diamos, et al., Deep speech 2: End-to-end speech recognition in english and mandarin, 2016, pp. 173–182.
- [294] T. Sercu, V. Goel, Advances in very deep convolutional neural networks for LVCSR, in: *Proceedings of the International Speech Communication Association (INTERSPEECH)*, 2016, pp. 3429–3433.
- [295] L. Tóth, Convolutional deep maxout networks for phone recognition., in: *Proceedings of the International Speech Communication Association (INTERSPEECH)*, 2014, pp. 1078–1082.
- [296] T.N. Sainath, B. Kingsbury, A. Mohamed, G.E. Dahl, G. Saon, H. Soltau, T. Beran, A.Y. Aravkin, B. Ramabhadran, Improvements to deep convolutional neural networks for LVCSR, in: *Proceedings of the Automatic Speech Recognition and Understanding (ASRU) Workshops*, 2013, pp. 315–320.
- [297] D. Yu, W. Xiong, J. Droppo, A. Stolcke, G. Ye, J. Li, G. Zweig, Deep convolutional neural networks with layer-wise context expansion and attention, in: *Proceedings of the International Speech Communication Association (INTERSPEECH)*, 2016, pp. 17–21.
- [298] A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, K.J. Lang, Phoneme recognition using time-delay neural networks, *IEEE Trans. Acoustics, Speech, Signal Process.* 37 (3) (1989) 328–339.
- [299] L.-H. Chen, T. Raitio, C. Valentini-Botinhao, J. Yamagishi, Z.-H. Ling, Dnn-based stochastic postfilter for hmm-based speech synthesis., in: *Proceedings of the International Speech Communication Association (INTERSPEECH)*, 2014, pp. 1954–1958.
- [300] B. Uria, I. Murray, S. Renals, C. Valentini-Botinhao, J. Bridle, Modelling acoustic feature dependencies with artificial neural networks: Trajectory-rnade, in: *Proceedings of the International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2015, pp. 4465–4469.
- [301] Z. Huang, S.M. Siniscalchi, C.-H. Lee, Hierarchical bayesian combination of plug-in maximum a posteriori decoders in deep neural networks-based speech recognition and speaker adaptation, *Pattern Recognit. Lett.* (2017).
- [302] A. van den Oord, N. Kalchbrenner, K. Kavukcuoglu, Pixel recurrent neural networks, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2016, pp. 1747–1756.
- [303] R. Jozefowicz, O. Vinyals, M. Schuster, N. Shazeer, Y. Wu, Exploring the limits of language modeling, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [304] Y. Kim, Y. Jernite, D. Sontag, A.M. Rush, Character-aware neural language models, in: *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI)*, 2016, pp. 2741–2749.
- [305] J. Gu, C. Jianfei, G. Wang, T. Chen, Stack-captioning: coarse-to-fine learning for image captioning, volume abs/1709.03376, 2017.
- [306] M. Wang, Z. Lu, H. Li, W. Jiang, Q. Liu, *gen cnn*: a convolutional architecture for word sequence prediction, in: *Proceedings of the Association for Computational Linguistics (ACL)*, 2015, pp. 1567–1576.
- [307] J. Gu, G. Wang, C. Jianfei, T. Chen, An empirical study of language CNN for image captioning, in: *Proceedings of the International Conference on Computer Vision (ICCV)*, 2017.
- [308] M.A.D.G. Yann N. Dauphin Angela Fan, Language modeling with gated convolutional networks, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2017, pp. 933–941.
- [309] R. Collobert, J. Weston, A unified architecture for natural language processing: deep neural networks with multitask learning, in: *Proceedings of the International Conference on Machine Learning (ICML)*, 2008, pp. 160–167.
- [310] L. Yu, K.M. Hermann, P. Blunsom, S. Pulman, Deep learning for answer sentence selection, in: *Proceedings of the Advances in Neural Information Processing Systems (NIPS) Workshop*, 2014.
- [311] N. Kalchbrenner, E. Grefenstette, P. Blunsom, A convolutional neural network for modelling sentences, in: *Proceedings of the Association for Computational Linguistics (ACL)*, 2014, pp. 655–665.
- [312] Y. Kim, Convolutional neural networks for sentence classification, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.
- [313] W. Yin, H. Schütze, Multichannel variable-size convolution for sentence classification, in: *Proceedings of the Conference on Natural Language Learning (CoNLL)*, 2015, pp. 204–214.
- [314] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, P. Kuksa, Natural language processing (almost) from scratch, *J. Mach. Learn. Res. (JMLR)* 12 (2011) 2493–2537.
- [315] A. Conneau, H. Schwenk, L. Barrault, Y. Lecun, Very deep convolutional networks for natural language processing, *CoRR abs/1606.01781* (2016).
- [316] G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Weinberger, Deep networks with stochastic depth, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016, pp. 646–661.

Jiuxiang Gu is currently a Ph.D. candidate at Interdisciplinary Graduate School, Nanyang Technological University. He received his M.S. degree in 2013 from the University of Chinese Academy of Sciences in 2013. His research interests include computer vision and natural language processing.

Zhenhua Wang is a Postdoctoral Research Fellow of Rapid-Rich Object Search Lab, Nanyang Technological University. Before this, He received the Ph.D. Degree from National Laboratory of Pattern Recognition, Institute of Automation, Chinese Academy of Sciences in 2014. His main research interests are in the fields of Computer Vision.

Jason Kuen is a third-year Ph.D. candidate at School of EEE, Nanyang Technological University. Before this, He was an undergraduate at Multimedia University, Malaysia. His research is about leveraging machine learning algorithms to tackle challenging computer vision problems.

Lianyang Ma is a Postdoctoral Research Fellow of Rapid-Rich Object Search Lab, Nanyang Technological University. He received his Ph.D. degree from Shanghai Jiao Tong University. His research is mainly focused on surveillance video analysis, computer vision and machine learning.

Amir Shahroudy is a Ph.D. candidate at Nanyang Technological University and Institute for Infocomm Research, Singapore, under A*STAR's SINGA scholarship. He received his masters in Artificial Intelligence from Sharif University of Technology, Iran, in 2006. His research interests include action recognition and deep learning.

Bing Shuai is currently a Ph.D. candidate in machine learning and computer vision at School of EEE, Nanyang Technological University, Singapore. He received the B.E. degree in computer science from Chongqing University, China, in 2010. From 2010 to 2013, he was a master student had studied in the area of computer vision in Xiamen University, China.

Ting Liu is currently a Ph.D. candidate at School of EEE, Nanyang Technological University, Singapore. His research is mainly focused on object tracking, computer vision and deep learning.

Xingxing Wang is a Project officer at the School of EEE, Nanyang Technological University, Singapore. His research is mainly focused on convolutional neural networks and deep learning.

Gang Wang is an Associate Professor in EEE at the Nanyang Technological University. He is also a Research Scientist at the Advanced Digital Science Center. He received the B.S. degree from Harbin Institute of Technology, China, in 2005 and the Ph.D. degree from the University of Illinois at Urbana-Champaign, Urbana. His research interests include computer vision and machine learning.

Jianfei Cai received his Ph.D. degree from the University of Missouri-Columbia. He is currently an Associate Professor and has served as the Head of Visual & Interactive Computing Division and the Head of Computer Communication Division at the School of Computer Engineering, Nanyang Technological University, Singapore. His major research interests include computer vision, visual computing and multimedia networking. He has been actively participating in program committees of various conferences. He has served as the leading Technical Program Chair for IEEE International Conference on Multimedia & Expo (ICME) 2012 and the leading General Chair for Pacific-rim Conference on Multimedia (PCM) 2012. Since 2013, he has been serving as an Associate Editor for IEEE Trans on Image Processing (T-IP). He has also served as an Associate Editor for IEEE Trans on Circuits and Systems for Video Technology (T-CSVT) from 2006 to 2013.

Tsuhhan Chen is currently the Dean of the College of Engineering, and the Cheng Tsang Man Chair Professor, at the Nanyang Technological University, Singapore. He received the B.S. degree in electrical engineering from the National Taiwan University, Taiwan, R.O.C., in 1987, and the M.S. and Ph.D. degrees in electrical engineering from the California Institute of Technology, Pasadena, CA, in 1990 and 1993, respectively. He founded the Technical Committee on Multimedia Signal Processing, and the Multimedia Signal Processing Workshop, both in the IEEE Signal Processing Society. His endeavor later evolved into the founding of the IEEE Transactions on Multimedia and the IEEE International Conference on Multimedia and Expo, both joining the efforts of multiple IEEE societies. He was appointed the Editor-in-Chief for IEEE Transactions on Multimedia in 2002–2004, during which the journal grew from 400 pages and 4 issues a year, to 1200 pages and 6 issues a year. Before serving as the Editor-in-Chief for IEEE Transactions on Multimedia, He also served on the Editorial Board of IEEE Signal Processing Magazine and as Associate Editor for IEEE Trans. on Circuits and Systems for Video Technology, IEEE Trans. on Image Processing, IEEE Trans. on Signal Processing, and IEEE Trans. on Multimedia. He co-edited a book titled Multimedia Systems, Standards, and Networks. He received the Charles Wilts Prize for outstanding independent research in Electrical Engineering leading to a Ph.D. degree at the California Institute of Technology in 1993. He was a recipient of the National Science Foundation CAREER Award, titled “Multimodal and Multimedia Signal Processing”, from 2000 to 2003. He received the Benjamin Richard Teare Teaching Award in recognition of excellence in engineering education at the Carnegie Mellon University in 2006, the Eta Kappa Nu Award for Outstanding Faculty Teaching in 2007, and the Michael Tien Teaching Award in 2014. He has published more than three hundreds of technical papers and holds 26 U.S. patents. He was elected to the Board of Governors, 2007–2009, and a Distinguished Lecturer, 2007–2008, both with the IEEE Signal Processing Society. He served as Vice President in 2012–2013, and as President in 2013–2014, of the ECE Department Head Association. He is a member of the Phi Tau Phi Scholastic Honor Society, and Fellow of IEEE.